University of Texas at Arlington

MavMatrix

Mavs Open Press Open Educational Resources | Mavs Open Press

8-19-2026

Computer Organization and Assembly Language Programming

Chenxi Wang PhD
University of Texas at Arlington

Muhammad Rashed PhD
University of Texas at Arlington

Follow this and additional works at: https://mavmatrix.uta.edu/oer_mavsopenpress2

 Part of the Programming Languages and Compilers Commons

Recommended Citation

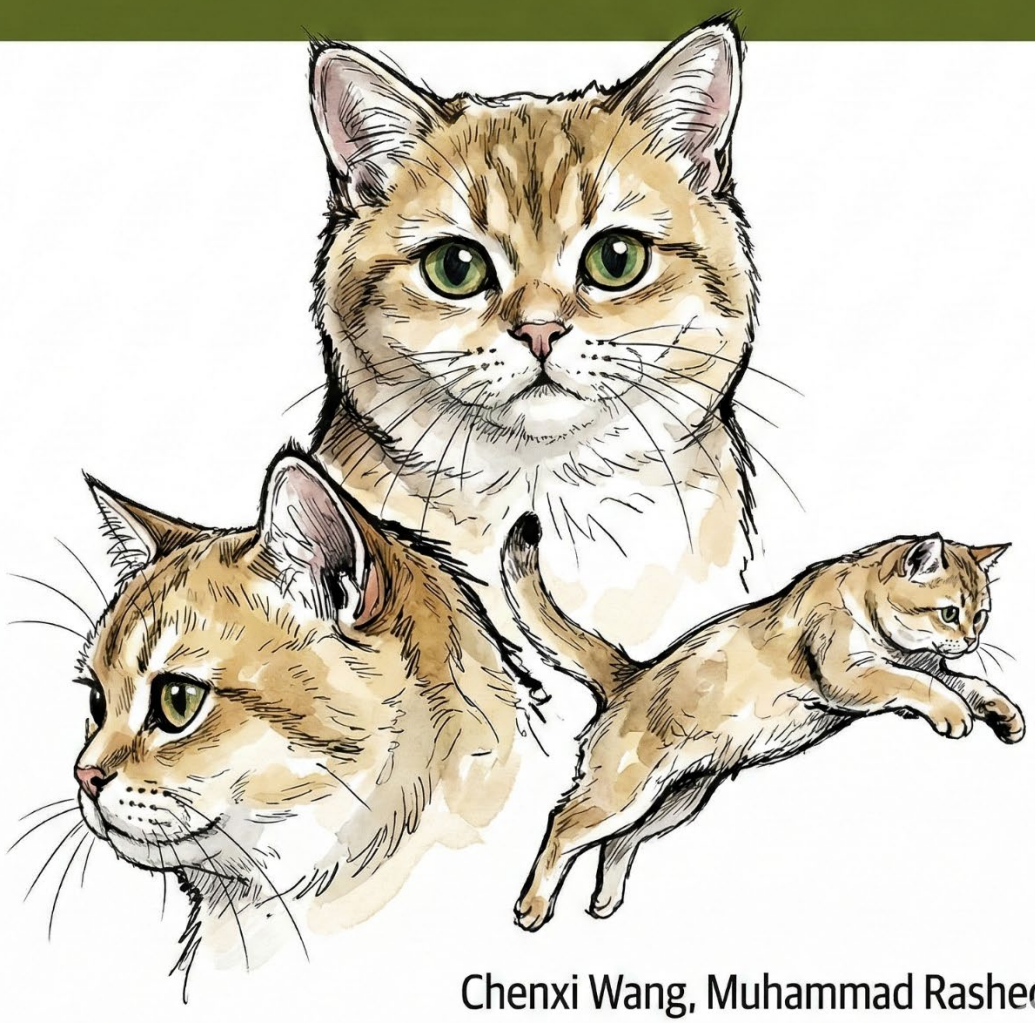
Wang, Chenxi PhD and Rashed, Muhammad PhD, "Computer Organization and Assembly Language Programming" (2026). *Mavs Open Press Open Educational Resources*. 3.
https://mavmatrix.uta.edu/oer_mavsopenpress2/3

This Book is brought to you for free and open access by the Mavs Open Press at MavMatrix. It has been accepted for inclusion in Mavs Open Press Open Educational Resources by an authorized administrator of MavMatrix. For more information, please contact vanessa.garrett@uta.edu.

UTA Open Educational Resources

1st Edition

COMPUTER ORGANIZATION AND ASSEMBLY LANGUAGE PROGRAMMING



Chenxi Wang, Muhammad Rashed

CONTENTS

Acknowledgement 9
About The Publisher 11
Accessibility Statement 12
Chapter 1. Introduction to CSE2312 13
1.1 What is Computer Organization? 13
1.2 Why Learn Assembly? 14
1.3 Course Goals and Learning Outcomes 15
Chapter 2. The Number Systems 16
2.1 Binary (Base-2) Number System 16
2.1.1 Unsigned Binary Numbers 17
2.1.2 Signed Binary Numbers 17
2.1.3 Two's Complement 18
2. Octal (Base-8) Number System 19
3. Decimal (Base-10) Number System 19
4. Hexadecimal (Base-16) Number System 19
Chapter 3. Conversion of Number Systems 21
3.1 Convert Decimal to Binary (Unsigned) 21
3.1.1 Method A: Weights Calculation to convert Decimal to Binary (Unsigned) 21
3.1.2 Method B: Division-by-2, to convert Decimal to Binary (Unsigned) 22
3.2 Convert Binary (Unsigned) to Decimal 22
3.3 Between Decimal and Hexadecimal (Hex) 23
Chapter 4. ALU Operations 25
4.1 The Role of the ALU 25
4.2 ALU Architecture 25
4.3 Arithmetic Operations 26
Addition 26
Negation and Two's Complement 26

Subtraction 26
4.4 Logical Operations 26
4.5 Bitwise Operations and Masking 26
Checking a Bit 27
Clearing a Bit 27
Setting a Bit 27
Chapter 5. Flags and Shift Operations 28
5.1 Status Flags 28
5.1.1 The Carry Flag (C) 28
5.1.2 The Zero Flag (Z) 29
5.1.3 The Negative Flag (N) 30
Meaning of the Negative Flag 30
5.1.4 The Overflow Flag (V) 30
Meaning of the Overflow Flag 30
5.1.5 Flag Summary and Examples 31
Practice Problems 31
5.2 Shift Operations 32
5.2.1 Logical Shift 33
Logical Shift Left (LSL) 33
Logical Shift Right (LSR) 33
5.2.2 Arithmetic Shift 33
Arithmetic Shift Left (ASL) 34
Arithmetic Shift Right (ASR) 34
5.2.2 Rotate Right (ROR) 34
Chapter 6. Assembly Syntax 35
6.1 Addition Instruction: ADD {cond}{S} Rd, Rn, <Oprnd2> 35
6.2 Introduction of the Condition Suffix {cond} 35
6.3 Introduction of Flag-setting Suffix {S} 37
6.4 Introduction of the Argument2 of ALU - <Oprnd2> 38
6.5 Addition with carry instruction: ADC{cond}{S} Rd, Rn, <Oprnd2> 39

6.6 Data move instruction: MOV{cond}{S} Rd, <Oprnd2> 40
6.7 MOV with Status Registers 40
6.8 Subtraction Instructions: SUB{S} Rd, Rn, Operand2 41
6.8.1 Syntax: SUB{S} Rd, Rn, Operand2 41
6.8.2 Syntax: SBC{S} Rd, Rn, Operand2 42
6.8.3 Syntax: RSB{S} Rd, Rn, Operand2 42
6.9 Additional Instructions 42
6.9.1 Syntax: CMP Rn, Operand2 42
6.9.2 Syntax: CMN Rn, Operand2 43
6.9.3 Syntax: MUL{S} Rd, Rn, Rm 43
6.9.4 Syntax: MLA{S} Rd, Rn, Rm, Ra 43
Chapter 7. Data in Memory and the Load-Store Architecture 44
7.1 The Role of Memory 44
7.2 Memory Organization 45
7.2.1 Byte-Addressable Memory 45
7.2.2 Pointers and Memory Addresses 46
7.3 Endianness: How Multi-Byte Data is Stored 46
7.3.1 Little-Endian 47
7.3.2 Big-Endian 47
7.4 ARMv7 Store Instructions 48
7.4.1 STRB: Store Byte 48
Example 48
7.4.2 STRH: Store Half-word 48
Example 48
7.4.3 STR: Store Word 49
Example 49
7.5 ARMv7 Load Instructions 49
7.5.1 LDRB: Load Unsigned Byte 49
Example 49
7.5.2 LDRSB: Load Signed Byte 49

Example 50
7.5.3 LDRH and LDRSH: Load Half-word 50
7.5.4 LDR: Load Word 50
7.6 Addressing Modes 50
7.6.1 Register Direct Addressing 50
7.6.2 Offset Addressing 51
7.6.3 Post-Indexed Addressing 51
Chapter 8. Branch Instructions, Procedure Calling Mechanism, and Assembly Implementation 52
8.1 Link Register and Program Counter Register 52
8.1.1 Register LR (R14): Link Register 52
8.1.2 Register PC (R15): Program Counter 52
8.2 Syntax of Branch Instructions 53
8.2.1 Syntax: B label 53
8.2.2 Syntax: BL label 53
8.2.3 Syntax: BX LR 53
8.2.4 Syntax: B[cond] label 54
8.3 Procedure Calling Mechanism 57
8.3.1 Role of Procedures 58
8.3.2 Role of the Link Register (LR) 58
8.3.3 Returning from a Procedure: 58
8.4 Flow of Control 58
8.5 Loop with Conditional Branch Instruction 59
8.5.1 Solution A: Shifting 59
8.5.2 Solution B: Masking 61
Chapter 9. Stack 65
Introduction to the Stack 65
9.1 Architecture and Registers 66
9.1.1 The Special Registers 66
9.1.2 Stack Growth Direction 66
9.2 PUSH and POP Mechanics 67

9.3 Function Calls: Branch with Link (BL) 68
9.4 Worked Example: Tracing Memory and Registers 68
Initial State: 68
The Code: 69
Step-by-Step Execution Trace: 69
Final Stack Memory Map: 69
9.5 Variable Scope: Stack vs. Data Segment 69
9.5.1 Local Variables (Stack) 69
9.5.2 Static Variables (Data Segment) 70
9.6 Summary 70
Chapter 10. String (Type Char) in Memory and Assembly Implementation 71
10.1 Type Char in Memory 71
10.2 String in Memory 71
10.3 Assembly Implementation 72
Chapter 11. Array of Integers in Memory and Assembly Implementation 77
11.1 Memory Layout: Strings vs. Integers 77
11.1.1 32-bit Integer Arrays (uint32_t) 77
11.1.2 16-bit Integer Arrays (uint16_t) 78
11.2 Iterating with Post-Indexed Addressing 78
11.3 Coding Examples 79
11.3.1 Array Summation (sumU32) 79
11.3.2 Scaling an Array (scaleU32) 79
11.4 Advanced Exercise: Dot Product (dotpU32) 80
Chapter 12. Binary Decoding and Encoding (Floating-point and Decimal) 81
12.1 Integer (recap) 81
12.1.1 From Binary to Decimal 81
12.1.2 From Decimal to Binary 81
12.2 Non-integer (Floating-point) 82
12.2.1 From Decimal to Binary 82
12.2.2 Decoding - From Binary to Decimal 84

12.2.3 Encoding - From Decimal to Binary 87
Chapter 13. Data Alignment (Packed and Unpacked) and Assembly Implementation 91
13.1 Unpacked Structure in Memory 92
13.2 Packed Structure in Memory 94
13.3 Comparison Between Unpacked and Packed Cases 94
Chapter 14. VFP Instructions (VLDR, VSTR, VMOV, and VCVT) and Assembly Implementation 96
14.1 The VFP Coprocessor 96
14.2 VFP Register Organization 96
14.2.1 Register Banks 96
14.2.2 Register Aliasing 96
14.3 Procedure Call Standard (PCS) for VFP 97
14.3.1 Argument Passing Rules 97
14.3.2 Mixed Argument Example 98
14.4 VFP Instructions 98
14.4.1 Load and Store (VLDR, VSTR) 98
14.4.2 Moving Data (VMOV) 99
14.4.3 Data Conversion (VCVT) 99
14.4.4 Arithmetic Operations 99
14.5 Coding Examples 100
14.5.1 Array Summation (sumF64) 100
14.5.2 Calculating Mean (meanF64) 100
Chapter 15. Clock Cycle and Pipeline in ARM 102
15.1 Clock Cycle 102
15.2 Three-Stage Execution of Instructions 102
15.3 Pipeline in ARM 103
15.3 Analysis of Clock Cycles 106
Chapter 16. Physical Memory (RAM), Cache, and Memory Virtualization 109
16.1 The Memory Bottleneck 109
16.2 The Principle of Locality 109
16.2.1 Temporal Locality 109

16.2.2 Spatial Locality....110
16.3 Cache Memory Hierarchy....110
16.3.1 The Hierarchy Levels....111
16.3.2 The Inclusive Subset Property....111
16.3.3 Data Movement Rules....111
16.4 Average Memory Access Time....112
16.4.1 Calculation Example....112
16.5 Memory Virtualization....112
16.5.1 The Problems with Physical RAM....112
16.5.2 The Solution: Virtual Memory....114
16.5.3 Benefits of Virtualization....115
16.5.4 Limitation of Virtualization....116
Image Credits....117
Ai Disclosure....118
Errata And Versioning History....119

Acknowledgement

UTA CARES GRANT PROGRAM

Creation of this OER was funded by the UTA CARES Grant Program, which is sponsored by UTA Libraries. Under the auspices of UTA's Coalition for Alternative Resources in Education for Students (CARES), the grant program supports educators interested in practicing open education through the adoption of OER and, when no suitable open resource is available, through the creation of new OER or the adoption of library-licensed or other free content. Additionally, the program promotes innovation in teaching and learning through the exploration of open educational practices, such as collaborating with students to produce educational content of value to a wider community. Information about the grant program and funded projects is available online.

AUTHOR'S NOTE

This textbook is the result of a collective effort to enhance the learning experience for students in CSE2312: Computer Organization and Assembly Language Programming. We would like to express our deepest gratitude to the following individuals and organizations whose support made this Open Education Resource (OER) possible:

- **UTA Libraries and the UTA CARES Grant Program**: We are sincerely grateful for the financial and technical support provided through the UTA CARES OER Creation Grant. The guidance provided by the OER & Digital Scholarship team and the Mavs Open Press was invaluable in navigating the world of open publishing and hosting this work on Pressbooks.
- **Department of Computer Science and Engineering**: We appreciate the ongoing support from our department leadership and colleagues. Their commitment to innovative teaching allowed us the flexibility to develop a resource tailored specifically to the needs of our curriculum.
- **Our Fellow CSE 2312 Instructors and GTAs**: We extend our sincere thanks to Dr. Dianqi Han, Dr. Jason H Losh, and all the course instructors and teaching assistants. Their real-time feedback, shared course materials, precious insights, and willingness to experiment with new pedagogical approaches have been essential in refining this content for clarity and practical application.
- **Our Students**: This book is, first and foremost, for you. We thank the students who provided feedback during the pilot semester. Your questions and engagement helped us identify the gaps in traditional texts and motivated us to create a more balanced, cohesive guide to computer architecture and assembly.

- **A more personal note from Chenxi**: A special *thank you* goes to my cat, *Dollar*, who you'll see on the book cover, for being my constant, quiet company (well, not always quiet) by my side and keeping me motivated through every long hour of writing this book.

Our goal was to create something truly helpful for you. As you dive into the fascinating world of computer organization, we hope this book makes your path through the complexities of computer systems a little smoother and a lot more enjoyable.

LEAD AUTHOR/EDITOR/PROJECT MANAGER

Dr. Chenxi Wang, Assistant Professor of Instruction and Director of Computer Engineering Program, Department of Computer Science and Engineering, The University of Texas at Arlington

CONTRIBUTING AUTHORS

Dr. Muhammad Rashed, Assistant Professor, Department of Computer Science and Engineering, The University of Texas at Arlington

ADDITIONAL THANKS TO

Dr. Dianqi Han, Assistant Professor, Department of Computer Science and Engineering, The University of Texas at Arlington

Dr. Jason H Losh, Professor of Practice, Department of Computer Science and Engineering, The University of Texas at Arlington

About The Publisher

If you are an instructor who is using this OER for a course, please let us know by filling out our OER Adoption Form.

ABOUT MAVS OPEN PRESS

Creation of this resource was supported by Mavs Open Press, operated by the University of Texas at Arlington Libraries (UTA Libraries). Mavs Open Press, operated by the University of Texas at Arlington Libraries (UTA Libraries), offers no-cost services for UTA faculty, staff, and students who wish to openly publish their scholarship. The Libraries' program provides human and technological resources that empower our communities to create or adapt open educational resources (OER). Course materials published by Mavs Open Press are openly licensed using Creative Commons licenses to allow for revision and reuse and are offered in various digital formats free of charge. Mavs Open Press also supports the publication of open-access journals authored and edited by UTA faculty, staff, and students.

ABOUT OER

OER are free teaching and learning materials that are licensed to allow for revision and reuse. They can be fully self-contained textbooks, videos, quizzes, learning modules, and more. OER are distinct from public resources in that they permit others to use, copy, distribute, modify, or reuse the content. The legal permission to modify and customize OER to meet the specific learning objectives of a particular course make them a useful pedagogical tool.

CONTACT US

Information about open education at UTA is available online. Contact us at digitalpublishing@uta.libanswers.com for other inquires related to UTA Libraries publishing services.

Accessibility Statement

UTA Libraries believe education needs to be available to everyone, which means supporting the creation of free, open, and accessible educational resources. We are actively committed to increasing the accessibility and usability of the OER we produce.

KNOWN ACCESSIBILITY ISSUES

The rubric used to evaluate this resource for accessibility is included in the publication's back matter. While we strive to make our resources as accessible and as usable as possible, we might not always get it right. Any issues we identify will be listed below. There are currently no known issues.

List of Known Accessibility Issues

Location of issue	Need for improvement	Timeline	Workaround

If you encounter problems accessing this resource, please contact us at digitalpublishing@uta.libanswers.com to let us know so we can address the issue.

Please include the following information:

- The location of the problem by providing a web address or page description
- A description of the problem
- The computer, software, browser, and any assistive technology you are using that can help us diagnose and solve your issue
    - e.g., Windows 10, Google Chrome (Version 65.0.3325.181), NVDA screenreader

This statement was last updated on February 24, 2020. It was modified from the BCcampus Open Education Accessibility Toolkit – 2nd Edition by Amanda Coolidge, Sue Doner, Tara Robertson, and Josie Gray and is used under a CC BY 4.0 International License.

Chapter 1. Introduction to CSE2312

This course introduces fundamental concepts in computer organization and assembly language programming. It bridges the gap between software and hardware, helping students understand how high-level code translates into machine operations. Mastering these concepts is essential for all students in computer science and engineering majors to help them prepare for advanced topics such as computer architecture, operating systems, embedded systems, and compiler design.

1.1 What is Computer Organization?

Computer organization refers to the structural and operational design of a computer system. It encompasses several core components that work together to execute tasks.

- **Central Processing Unit (CPU)**
- **Memory (RAM & Storage)**
- **Bus System**
- **I/O Devices**

The Central Processing Unit (CPU) serves as the brain of the computer, executing instructions and processing data. The memory system, which includes RAM and storage, is responsible for storing programs and data. To facilitate communication between these components, the bus system transfers data across the architecture. Finally, Input/Output (I/O) devices enable the system to interact with the external physical world. Understanding these hardware components is crucial for programmers aiming to write efficient and optimized code.

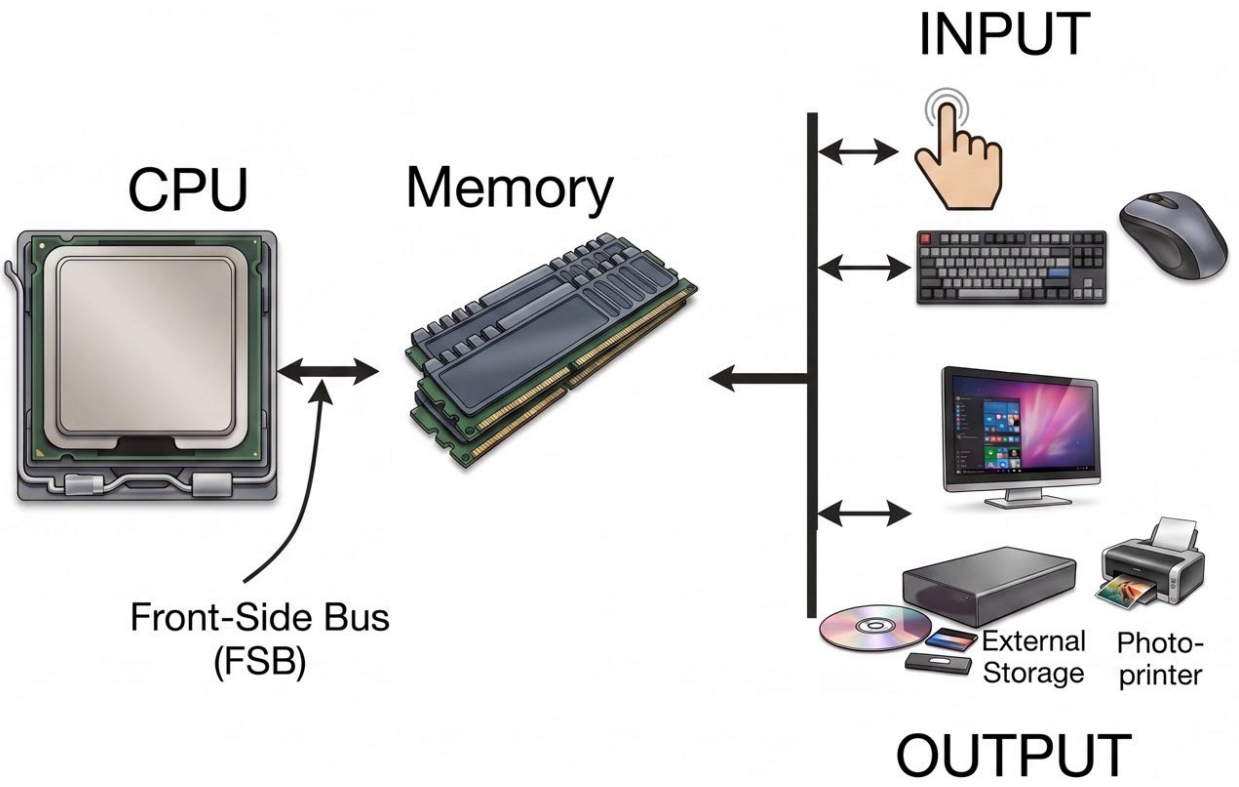


Fig 1.1: Components of Computers

1.2 Why Learn Assembly?

Assembly language is a low-level programming language that provides direct control over a computer's hardware. While most modern programming is conducted in high-level languages like C, Python, or Java, assembly programming remains highly relevant for several critical reasons. It offers unparalleled accessibility to system hardware, making it heavily utilized in system programming and embedded systems. Furthermore, assembly provides a high degree of efficiency, allowing developers to meticulously optimize for speed and memory usage. It also enhances debugging capabilities by helping programmers understand exactly how high-level code interacts with the underlying hardware. Consequently, assembly programming is crucial for applications requiring high performance, precise timing, and low-level hardware interactions.

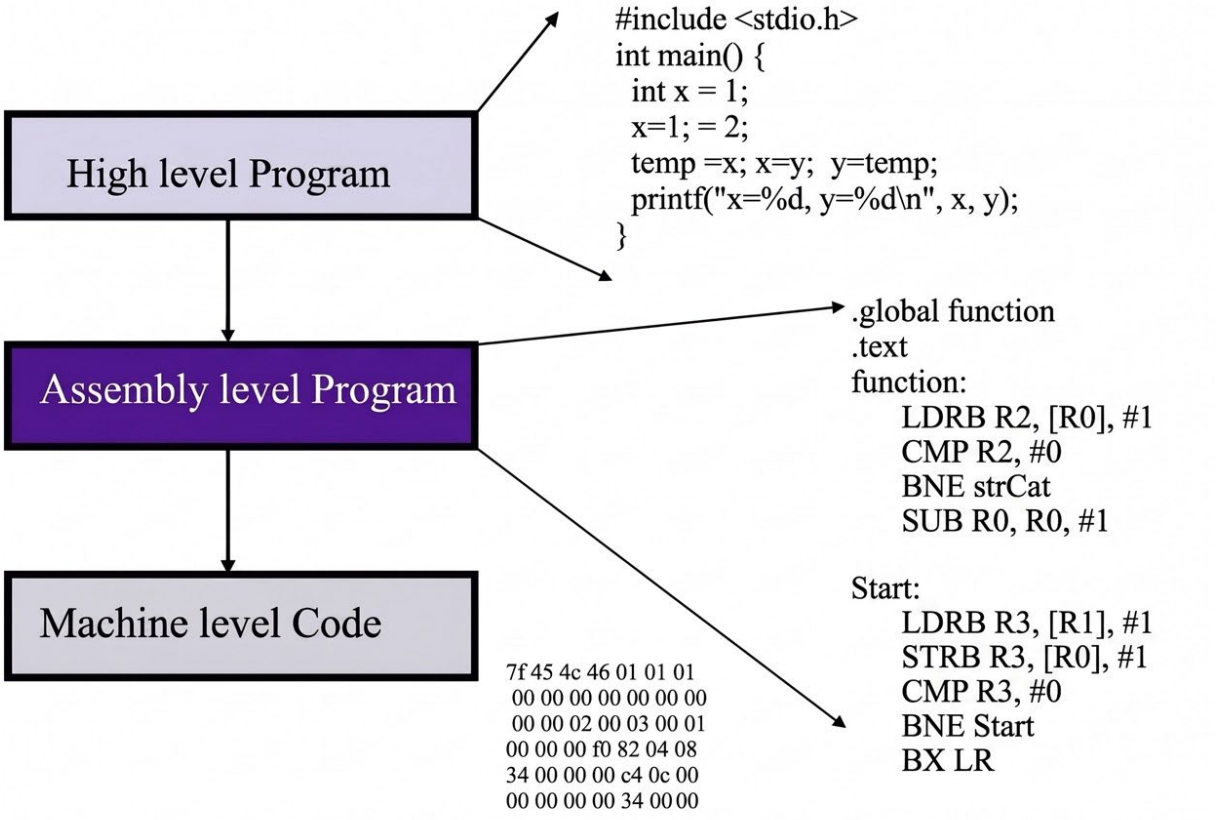


Fig 1.2: High-Level Language vs. Low-Level Language

1.3 Course Goals and Learning Outcomes

By successfully completing this course, students will thoroughly understand the internal structure of computers and learn how instructions are executed by processors. Students will develop practical skills in writing assembly language programs, gain deep insights into system performance optimization, and become proficient in debugging and analyzing low-level code. Ultimately, the course will explore the complex relationship between high-level programming languages and their assembly counterparts.

Chapter 2. The Number Systems

Different bases of the number systems in computer system architecture. In this course, we will explore the four main bases for the number systems in the realm of computer organization and assembly language programming.

Computer architecture supports following number systems:

- Binary (Base-2) Number System
- Octal (Base-8) Number System
- Decimal (Base-10) Number System
- Hexadecimal (Base-16) Number System

In this chapter, we will learn these four number systems.

Number System	Base	Digits Used	Prefix / Notation	Example
Binary	2	0, 1	0b	0b 0101 0101
Octal	8	0 – 7	0o (or leading 0)	0o 125
Decimal	10	0 – 9	(none)	85
Hexadecimal	16	0 – 9, A – F	0x	0x 55

Table 2.1 — The four number systems used in computer organization and assembly programming.

2.1 Binary (Base-2) Number System

As known as Binary Number System, the base-2 number system only has two digits, 0 and 1. Every number could be represented by 0 and 1 in this number system. As the notation of binary number, the prefix 0b is widely used in languages such as C, C++, Python, and JavaScript.

As a standard convention in documentation, spacing is used in our course for readability and convenience. Since binary numbers can be long and difficult to interpret immediately, dividing them into several 4-bit groups makes them easier to read and understand. Grouping helps in distinguishing different parts of the number.

For example, 0b 0101 0101, 0b 1111, and 0b 0000 0000 1111 0000 1111 0000 1111 0000 are all binary numbers with various bits. Additionally, we use the prefix 0b to indicate that the number following is written in binary (base-2) format.

It is important to note that most assemblers do not support direct spacing within binary literals in the actual code. However, comments can be used to group bits for improved readability.

In this course, we will learn two types of binary number representations: Unsigned and Signed.

2.1.1 Unsigned Binary Numbers

Unsigned Binary Numbers are used to represent only non-negative integers (zero and positive numbers).

For example:

- 8-bit unsigned binary 0b 0000 0001 equals to 1 in decimal
- 4-bit unsigned binary 0b 1111 equals to 15 in decimal

For an n-bit Unsigned binary number, the range is:

$$0 \text{ to } 2^n - 1$$

For example, an 8-bit unsigned binary number has a minimum value of 0b 0000 0000 and a maximum value of 0b 1111 1111, the range is 0 to 2^8, which is 0 to 255 in decimal.

Bit position	**7**	**6**	**5**	**4**	**3**	**2**	**1**	**0**
Weight	128	64	32	16	8	4	2	1
Example: 0b 0011 0101	0	0	1	1	0	1	0	1
Contribution	0	0	32	16	0	4	0	1

Figure 2.1 — Place-value chart for an 8-bit unsigned binary number. Each bit contributes its weight when set to 1: the example 0b 0011 0101 equals 32 + 16 + 4 + 1 = 53 in decimal.

Width n (bits)	**Range (0 to $2^n - 1$)**	**Minimum**	**Maximum**
4	0 to $2^4 - 1$	0	15
8	0 to $2^8 - 1$	0	255
16	0 to $2^{16} - 1$	0	65,535
32	0 to $2^{32} - 1$	0	4,294,967,295

Table 2.2 — Range of n-bit unsigned binary numbers for common widths.

2.1.2 Signed Binary Numbers

Signed Binary Numbers represent both positive and negative values. We take the leftmost bit (MSB – Most Significant Bit) as the Sign bit of the unsigned binary numbers. If the sign bit is 0, then the binary number is positive. If the sign bit is 1, the binary number represents a negative value.

b7 (MSB)	b6	b5	b4	b3	b2	b1	b0
Sign bit	Magnitude	Magnitude	Magnitude	Magnitude	Magnitude	Magnitude	Magnitude
0: positive 1: negative	value bits	value bits	value bits	value bits	value bits	value bits	value bits

Figure 2.2 — Bit layout of an 8-bit signed binary number. The most significant bit (MSB) serves as the sign bit: 0 indicates a positive value, 1 indicates a negative value.

For example:

- 8-bit signed binary 0b 0000 0001 equals to 1 in decimal. Because the MSB (sign bit) is 0.
- 8-bit signed binary 0b 1111 1111 equals to -1 in decimal. Because the MSB (sign bit) is 1.
- 8-bit signed binary 0b 1000 0001 equals to -127 in decimal. Because the MSB (sign bit) is 1.
- 4-bit signed binary 0b 0010 equal to 2 in decimal.

For an n-bit Signed binary number, the range is:

$$-2^{n-1} \text{ to } 2^{n-1} - 1$$

For example, an 8-bit signed binary number has a minimum value of 0b 1000 0000 and a maximum value of 0b 0111 1111, the range is -2^{8-1} to 2^{8-1} -1, which is -128 to 127 in decimal.

Width n (bits)	Range (-2^{n-1} to $2^{n-1} - 1$)	Minimum	Maximum
4	-2^3 to $2^3 - 1$	-8	7
8	-2^7 to $2^7 - 1$	-128	127
16	-2^{15} to $2^{15} - 1$	-32,768	32,767
32	-2^{31} to $2^{31} - 1$	-2,147,483,648	2,147,483,647

Table 2.3 — Range of n-bit signed binary numbers for common widths.

2.1.3 Two's Complement

Two's complement is a mathematical operation on binary numbers to represent signed integers. It allows both positive and negative integers to coexist in a binary system while simplifying the arithmetic operations. Specifically, the general arithmetic operations (addition, subtraction, and multiplication) work seamlessly without requiring separate handling for negative numbers.

In addition, the other advantage of Two's complement is efficient hardware implementation. Because the same binary adder circuit can be used for both signed and unsigned operations, there is no need for a separate subtraction circuit. The subtraction can be simply performed by the addition of a negative number.

Two's complement is a standard representation adopted by most computers and digital systems to store signed numbers. We perform Two's completement operation by flipping all the bits and adding 1.

For example, to find the Two's complement of 8-bit signed binary 0b 0101 1100 by the following two steps.

- Step1. Inverting all the 8 bits of the signed binary 0b 0101 1100, resulting in 0b 1010 0011.
- Step2. Adding 0b 0000 0001 (1 in decimal) to the result of Step1. Then we obtained 0b 1010 0100, which represents -92 in decimal.
- We call the value 0b 1010 0100 is the Two's complement of 0b 0101 1100.

Step	Operation	Result	Decimal Meaning
Start	Original 8-bit value	0b 0101 1100	+92
Step 1	Invert every bit (one's complement)	0b 1010 0011	—
Step 2	Add 1	0b 1010 0100	−92

Figure 2.3 — Computing the two's complement of 0b 0101 1100: invert all bits, then add 1. The result 0b 1010 0100 represents −92 in decimal.

2. Octal (Base-8) Number System

Octal number system has eight digits from 0 to 7. Every number could be represented by these eight digits (0, 1, 2, 3, 4, 5, 6, 7) in this number system.

3. Decimal (Base-10) Number System

Decimal number system has ten digits from 0 to 9. Every number could be represented by these ten digits (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) in this number system.

4. Hexadecimal (Base-16) Number System

Hexadecimal (Hex) number system has sixteen digits from 0 to F. Every number could be represented by these sixteen digits (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F) in this number system. A is 10, B is 11, C is 12, D is 13, E is 14, and F is 15

Decimal	Binary (4-bit)	Octal	Hexadecimal
0	0b 0000	0	0x 0
1	0b 0001	1	0x 1
2	0b 0010	2	0x 2
3	0b 0011	3	0x 3
4	0b 0100	4	0x 4
5	0b 0101	5	0x 5
6	0b 0110	6	0x 6
7	0b 0111	7	0x 7
8	0b 1000	10	0x 8
9	0b 1001	11	0x 9
10	0b 1010	12	0x A
11	0b 1011	13	0x B
12	0b 1100	14	0x C
13	0b 1101	15	0x D
14	0b 1110	16	0x E
15	0b 1111	17	0x F

Table 2.4 — Equivalent values in the four number systems (0–15). Note how a single hexadecimal digit maps exactly to a 4-bit binary group, and highlighted cells show the hexadecimal digits A–F.

Chapter 3. Conversion of Number Systems

In this chapter, we will explore the different methods of conversion between number systems.

3.1 Convert Decimal to Binary (Unsigned)

3.1.1 Method A: Weights Calculation to convert Decimal to Binary (Unsigned).

Before we get start with the method, let's take a look what is the Weight. In decimal, the weight of the n-th digit from the rightmost is calculated as10^{n-1}.

For example, the weight of '7' in decimal 5732 is 10^2, because the number '7' is the 3rd digit from the rightmost of 5732.

- **Step 1:** Check whether the decimal number is within the range.
- **Step 2:** List the weights of bits.
- **Step 3:** Compare the decimal number with weights along the direction from MSB to LSB. If the number is larger than or equal to the weight, subtract the weight from the number, put '1' at this bit, and continue with the difference. Otherwise, put '0' at this bit and move on to the next bit on the right.

For example, convert the decimal 53 to binary, assuming 53 is uint8 data type.

Step 1: Check 53 is within the range of uint8 (range: 0 to 255).

Step 2: List the weights of bits.

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
128	64	32	16	8	4	2	1

Step 3:

53 ≥ 128?	53 ≥ 64?	53 ≥ 32?	21 ≥ 16?	5 ≥ 8?	5 ≥ 4?	1 ≥ 2?	1 ≥ 1?
No	No	Yes	Yes	No	Yes	No	Yes
		53-32=21	21-16=5		5-4=1		1-1=0
⇩	⇩	⇩	⇩	⇩	⇩	⇩	⇩
0	0	1	1	0	1	0	1

Therefore, the binary representation of given decimal 53 is 0b**00110101**.

3.1.2 Method B: Division-by-2, to convert Decimal to Binary (Unsigned).

- **Step 1.** Divide the decimal number by 2, record the remainder (0 or 1).
- **Step 2.** Take the quotient and continue divide it by 2.
- **Step 3.** Repeat the division until quotient is 0. Then the binary result is the remainders reading in reverse order.

For example, convert the decimal number 67 to binary.

Step 1 and Step 2:

67 ÷ 2 = 33, remainder **1**.

33 ÷ 2 = 16, remainder **1**.

16 ÷ 2 = 8, remainder **0**.

8 ÷ 2 = 4, remainder **0**.

4 ÷ 2 = 2, remainder **0**.

2 ÷ 2 = 1, remainder **0**.

1 ÷ 2 = 0, remainder **1**.

Step 3:

The binary result is the remainders reading in **reverse** order: 1, 0, 0, 0, 0, 1, 1.

Therefore, the binary representation of 67 is 0b**1000011.** For 8-digit binary format: 0b**01000011**. In binary, leading zeros don't affect the value of the number, so they can be added or removed as needed for formatting purposes.

3.2 Convert Binary (Unsigned) to Decimal

We can calculate the sum of all weights to convert a binary (unsigned) to decimal.

- **Step 1.** List the weights of all the bits.
- **Step 2.** Multiply bits by their corresponding weights.
- **Step 3.** Calculate the sum of all products.

For example, uint8_t x = 0b10101010, what's the decimal of x?

Step 1: List the weights of all the bits.

binary:	1	0	1	0	1	0	1	0
weights:	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

Step 2: Multiply bits by their corresponding weights.

$(1 \times 2^7) + (0 \times 2^6) + (1 \times 2^5) + (0 \times 2^4) + (1 \times 2^3) + (0 \times 2^2) + (1 \times 2^1) + (0 \times 2^0)$

Step 3: Calculate the sum of all products.

$= 128 + 32 + 8 + 2$

$= 170$

Thus, 170 is the decimal of binary 0b10101010.

3.3 Between Decimal and Hexadecimal (Hex)

To convert a Hexadecimal (Hex) to Decimal, we are following these three steps:

- **Step 1**: List the weights of all the digits
- **Step 2**: Multiple digits by their corresponding weights
- **Step 3**: Calculate the sum of all products

For example, convert ABCD (base-16) to Decimal

0x	A	B	C	D
Weight	16^3	16^2	16^1	16^0

Sum of all products = A x 4096 + B x 256 + C x 16 + D x 1

= 10 x 4096 + 11 x 256 + 12 x 16 + 13 x 1

= 43981

To convert a decimal to Hex, we use the method of Division-by-16, which is similar as Division-by-2.

- **Step 1.** Divide the decimal number by 16, record the remainder (0 or 1).
- **Step 2.** Take the quotient and continue divide it by 16.
- **Step 3.** Repeat the division until quotient is 0. Then the binary result is the remainders reading in reverse order.

For example, convert the decimal number 345 to Hex.

Step 1 - Divide 345 by 16

Quotient: 345÷16=21

Remainder: **9**

Step 2 - Divide 21 by 16

Quotient: 21÷16=1

Remainder: **5**

Step 3 - Divide 1 by 16

Quotient: 1÷16=0, so we stop here.
Remainder: **1**

Step 3:

The binary result is the remainders reading in **reverse** order: 1, 5, 9

Therefore, the hexadecimal representation of the decimal number 345 is 0x159.

Chapter 4. ALU Operations

4.1 The Role of the ALU

The Arithmetic Logic Unit (ALU) is a core element of the Central Processing Unit (CPU). It handles the execution of arithmetic and bitwise logical operations on binary numbers. Functioning as a combinational digital circuit, the ALU takes input values, applies specific mathematical or logical rules, and outputs the calculated result.

4.2 ALU Architecture

An ALU is built with specific hardware components. First, it receives two input operands, generally called Arg1 and Arg2. Next, an operation selector tells the unit which internal logic path to activate, such as addition or an AND operation. Finally, it includes a result output port and internal circuits made specifically for math and logic tasks.

The unit is built to process operations based on N-bit inputs. This N value relies on the specific processor design, like a 32-bit or 64-bit architecture. Since it uses combinational logic gates instead of sequential steps, these computations are completed extremely fast in just one clock cycle.

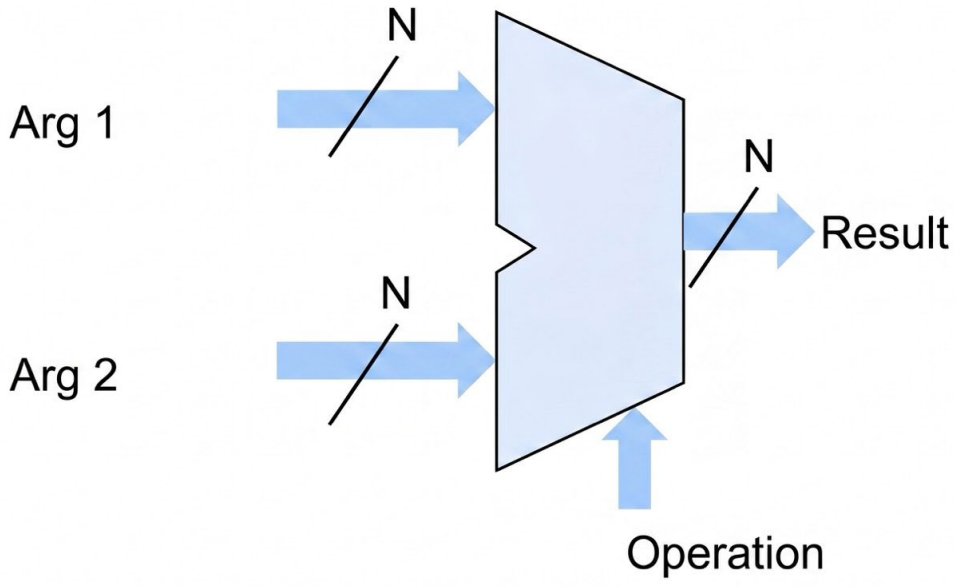


Fig 4.1: ALU Architecture

4.3 Arithmetic Operations

Various basic math operations are supported by the ALU, including standard addition, subtraction, and negation. All of these strictly obey binary math rules. Additionally, the hardware can perform multiplication through a sequence of optimized shift and add steps.

Addition

Adding binary numbers works a lot like decimal addition, carrying over a 1 to the next column when the sum is greater than 1. For instance, adding binary 0101 (which is 5) and 0011 (which is 3) gives 1000 (which is 8).

```
  0101 (5)
+ 0011 (3)
  ------
  1000 (8)
```

Negation and Two's Complement

To make a number negative, the ALU utilizes the two's complement approach. This technique flips all the bits of the starting number and adds 1. To negate 0101 (5), the bits become 1010; adding 1 yields 1011, representing -5.

Subtraction

Instead of building separate subtraction circuits, the ALU cleverly reuses its addition hardware. It does this by adding the two's complement of the second number to the first number. To find 5 - 3, it takes the two's complement of 3 and adds it to 5 to get 0010 (2).

4.4 Logical Operations

Bitwise logical instructions modify specific bits inside binary values by comparing corresponding bits of two inputs. The ALU offers several core logical functions:

- **AND:** Outputs a 1 only if both of the corresponding bits are 1. Example: 1101 & 1011 = 1001.
- **OR:** Outputs a 1 if at least one corresponding bit is 1. Example: 1101 | 1011 = 1111.
- **XOR (Exclusive OR):** Outputs a 1 only when the corresponding bits are different. Example: 1101 ^ 1011 = 0110.
- **NOT:** This is a unary operation that flips all the bits of a single operand, changing 1s to 0s and 0s to 1s. Example: ~1101 = 0010.

4.5 Bitwise Operations and Masking

A frequent use case for logical instructions in assembly is bit masking. Masks let developers isolate, modify, keep, or clear particular bits without changing the rest of the number.

Checking a Bit

To see if a specific bit is currently a 1, apply the AND operation using a mask where only your target bit is 1. If the result is 0, the bit was clear. If it is not zero, the bit was set.

```
x      = 0b11000111
 mask   = 0b00000100
 result = x & mask
 if result == 0: bit is clear
 else:           bit is set
```

Clearing a Bit

To guarantee a specific bit becomes 0, apply the AND operation with a mask that has a 0 at the target position and 1s everywhere else.

```
x      = 0b11010110
 mask   = 0b11111011
 result = x & mask
```

Setting a Bit

To guarantee a specific bit becomes a 1, apply the OR operation using a mask where the target bit is 1 and all other bits are 0.

```
x      = 0b11010110
 mask   = 0b00000100
 result = x | mask
```

Chapter 5. Flags and Shift Operations

5.1 Status Flags

After the ALU performs an operation, it's often necessary to know more about the result than just its value. For instance, did the operation result in a value of zero? Did it result in a negative number? Or, did the operation produce a result that was too large to be stored in the available number of bits?

To answer these questions, the ALU uses a set of special single-bit units called flags. These flags are stored in a status register (or flag register). The four main flags we will discuss are:

- Carry Flag (C)
- Zero Flag (Z)
- Negative Flag (N)
- Overflow Flag (V)

The hardware will always calculate and set these flags after an operation. However, the meaning of a particular flag often depends on whether we are working with signed or unsigned numbers.

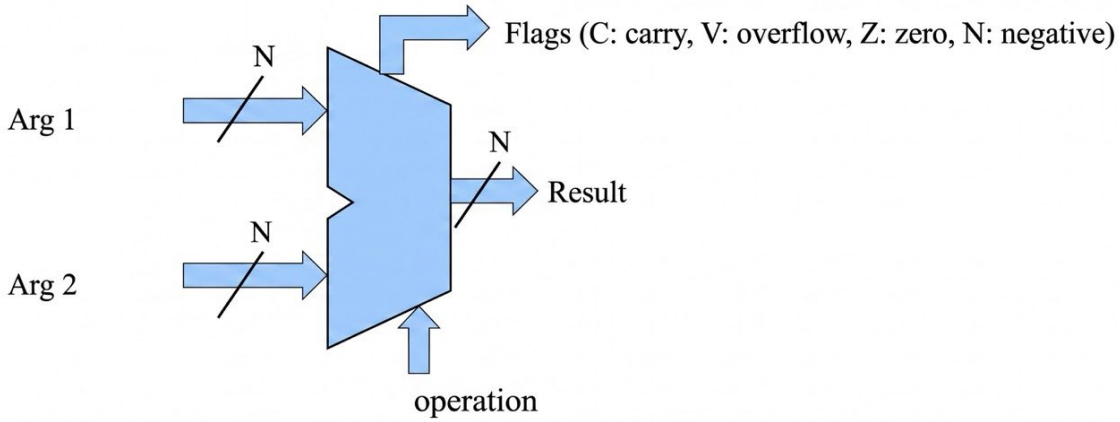


Fig 5.1: ALU and Flags

5.1.1 The Carry Flag (C)

The Carry Flag is set when an arithmetic operation generates a carry-out from the most significant bit (MSB).

Condition	Carry Flag (C)

Carry from the MSB?	
Yes	1 (Set)
No	0 (Clear)

Table 5. 1 Condition for Carry from the MSB and Carry Flag

Meaning of the Carry Flag

- **For Unsigned Operations:** The C flag is crucial. It indicates that the result of the operation exceeded the maximum value for the given number of bits. The carry bit can be thought of as an extra bit of the result, with a weight double that of the MSB.
- **For Signed Operations:** The C flag has no meaning for signed arithmetic. Overflow in signed numbers is detected by the V flag.

Example: Consider an 8-bit addition.

```
   01111111    (127)
 + 00000001    (1)

------------------
   10000000    (128)
```

No carry from MSB -> C = 0

```
  10000000  (128)
+ 10000000 (128)

1 00000000 (Result is 0, with a carry)
```

Carry from MSB -> C = 1

5.1.2 The Zero Flag (Z)

The Zero Flag is set if the result of an operation is zero (i.e., all bits of the result are 0).

Condition	**Zero Flag (Z)**
Are all bits in the result 0?	
Yes	1 (Set)
No	0 (Clear)

Table 5. 2 Condition for Zero Flag and its value

The Z flag is meaningful for both signed and unsigned operations.

Example: In the second example above, '10000000 + 10000000', the 8-bit result is '00000000'. Therefore, the Z flag would be set to 1.

5.1.3 The Negative Flag (N)

The Negative Flag (also called the Sign Flag) is a direct copy of the most significant bit (MSB) of the result.

Condition	Negative Flag (N)
MSB of result is 1?	1 (Set)
MSB of result is 0?	0 (Clear)

Table 5. 3 Condition for Negative Flag and its value

Meaning of the Negative Flag

- For Unsigned Operations: The N flag is meaningless, as unsigned numbers are always positive.
- For Signed Operations: If the Overflow Flag (V) is 0, the N flag correctly indicates the sign of the result. If N=1, the result is negative. If N=0, the result is positive or zero.

5.1.4 The Overflow Flag (V)

The Overflow Flag is set when the result of an arithmetic operation on signed numbers is too large to fit in the given number of bits, leading to an incorrect result. This is different from the carry flag.
Overflow can be detected by examining the signs of the operands and the result.

Operation	MSB of Arg1	MSB of Arg2	MSB of Result
Addition	1	1	0
Addition	0	0	1
(Otherwise)			0

Table 5. 4 Operations and its values for Overflow Flag

In simpler terms, for addition, an overflow occurs if:

- You add two positive numbers and get a negative result.
- You add two negative numbers and get a positive result.

Meaning of the Overflow Flag

- For Unsigned Operations: The V flag is meaningless.

- For Signed Operations: The V flag is the sole indicator of an overflow error. If V=1, the result is mathematically incorrect

Important Note: Logical operations (AND, OR, XOR, NOT) do **not** affect the V flag.

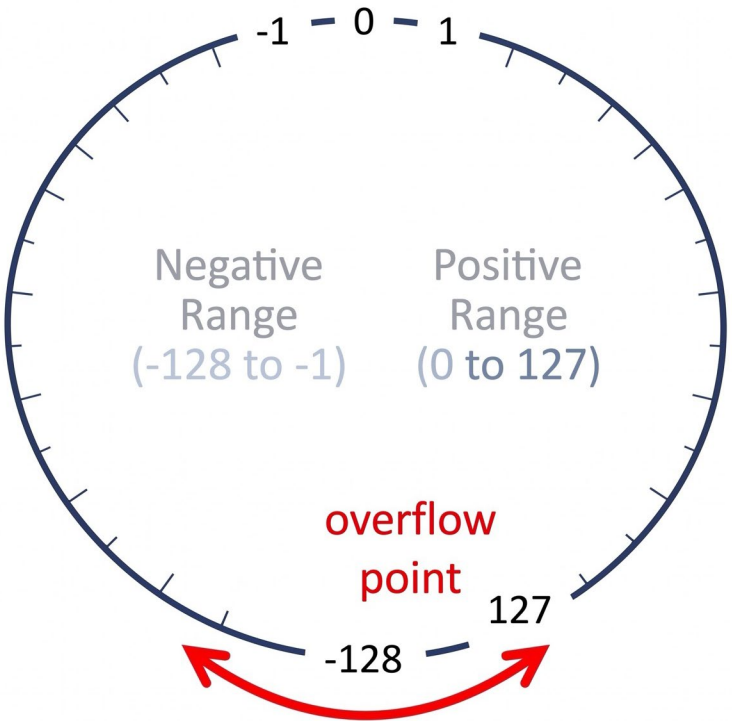


Fig 5.2: Visualization of overflow for 8-bit arithmetic

5.1.5 Flag Summary and Examples

Unsigned Integer	**Signed Integer**
Z: Result is all zeros	Z: Result is all zeros
C: Carry from MSB (indicates unsigned overflow)	C: Means nothing
N: Means nothing	N: Result is negative (if V=0)
V: Means nothing	V: Overflow error occurred

Table 5. 5 Flag Summary and Examples

Practice Problems

Let's analyze the flags for a few 8-bit operations.

Problem 1: Unsigned Addition

uint8_t x = 128; uint8_t y = 128; uint8_t z = x + y;

```
   10000000 (x)
+ 10000000 (y)
 100000000 (Result is 00000000, Carry is 1)
```

- C = 1 : There was a carry out of the MSB.
- Z = 1 : The 8-bit result is all zeros.
- N = 0 : The MSB of the 8-bit result is 0.
- V = 1 : We added two numbers with MSB=1 and got a Result MSB=0. (Note: V is meaningless for unsigned numbers).

Problem 2: Signed Addition

int8_t x = -128; int8_t y = -1; int8_t z = x + y;

```
  10000000 (x = -128)
+ 11111111 (y = -1 in 2's complement)
 101111111 (Result is 01111111, Carry is 1)
```

- C = 1 : There was a carry. (Meaningless for signed).
- Z = 0 : The result is not all zeros.
- N = 0 : The MSB of the result is 0.
- V = 1 : We added two negative numbers (MSB=1) and got a positive result (MSB=0). This indicates a signed overflow.

5.2 Shift Operations

Shift operations manipulate the bits of an operand by moving them to the left or right. They are fundamental operations used for multiplication, division, and bit-level data manipulation. The behavior of a shift operation depends on whether the operand is treated as a signed or unsigned number.

- **Logical Shift:** Used for unsigned numbers.
- **Arithmetic Shift:** Used for signed numbers.

5.2.1 Logical Shift

Logical shifts are used for unsigned binary numbers.

Logical Shift Left (LSL)

A logical shift left moves each bit one position to the left.

- The original MSB is moved into the Carry Flag (C).
- The least significant bit (LSB) position is filled with a 0.

A single LSL operation is equivalent to **multiplication by 2**. Shifting left by n bits is equivalent to multiplying by 2^n.

Example:

uint8_t x = 4; y = x << 1;

x = 00000100 (4)
C ← 0 ← [00001000] ← 0
y = 00001000 (8)

Logical Shift Right (LSR)

A logical shift right moves each bit one position to the right.

- The original LSB is moved into the Carry Flag (C).
- The MSB position is filled with a 0.

A single LSR operation is equivalent to **unsigned integer division by 2**.

Example:

uint8_t x = 4; y = x >> 1;

x = 00000100 (4)
0 → [00000010] → 0 → C
y = 00000010 (2)

5.2.2 Arithmetic Shift

Arithmetic shifts are used for signed binary numbers and are designed to preserve the number's sign.

Arithmetic Shift Left (ASL)

An arithmetic shift left is **identical** to a logical shift left. It moves each bit to the left, shifts the MSB into the Carry flag, and fills the LSB with a 0. It is equivalent to multiplication by 2. Because ASL and LSL are identical, most assembly languages, including ARM, only provide one instruction, typically 'LSL'.

Arithmetic Shift Right (ASR)

An arithmetic shift right is where the key difference lies. It moves each bit one position to the right, but it preserves the sign of the number.

- The original LSB is moved into the Carry Flag (C).
- The MSB position is filled with a copy of the original MSB. This is called sign extension. An ASR operation is equivalent to signed integer division by 2.

Example:
int8_t x = -2; y = x >> 1;

x = 11111110 (-2 in 2's complement)
1 → [11111111] → 0 → C
y = 11111111 (-1)

Notice how the MSB (which was 1) was copied into the new MSB position, preserving the negative sign of the number.

5.2.2 Rotate Right (ROR)

ARM also supports a rotate operation. Unlike a logical shift right that drops bits off the edge and fills the MSB with zeros, Rotate Right (ROR) takes the bits that fall off the right side (the LSBs) and feeds them directly back into the left side (the MSB positions).

Chapter 6. Assembly Syntax

In this chapter, we will explore the fundamental assembly syntax based on a specific ARM architecture - ARMv7, which is to be used as the reference in the entire book. The PDF version of Technical Reference Manual of ARMv7 (TRM ARMv7) could be find at the Appendix.

6.1 Addition Instruction: ADD {cond}{S} Rd, Rn, <Oprnd2>

ADD: Addition operation

- **{cond}**: Optional condition suffix. More details on Table 1-6 (Page 39 on TRM ARMv7)
- **{S}**: Optional suffix (with S, Flags changes according to the result; without S, the instruction does not affect Flags)
- **Rd**: destination register (where the result goes)
- **Rn**: the value in Rn is used as Arg 1 of ALU
- **<Oprnd2>**: Argument 2 of ALU. More details on Table 1-4 (Page 38 on TRM ARMv7)

6.2 Introduction of the Condition Suffix {cond}

In ARMv7, many instructions can be conditionally executed based on the condition suffix {cond}, which refers to the state of the condition flags in the Program Status Register (CPSR). This allows for efficient branching and decision-making without explicit branch instructions.

Example:

ADDEQS R0, R1, R2 LSL #2

- **ADD**: apply ADD operation
- **EQ**: if Z flag set (otherwise this instruction is not executed)
- **S**: the flags will be changed according to the result of this operation
- **R0**: write the result of this operation to R0
- **R1**: use R1 as Arg 1
- **R2**: use R2 LSL #2 as Arg 2

Assume R1 = 0x00000005, R2 = 0x00000003, Z = 1. What's the value of R0?

Step 1. Check the Condition (EQ):
The instruction will only execute if the Z flag is set (Z = 1). If Z = 0 (meaning the result of the previous instruction was not zero), this instruction is not executed.

Step 2. Perform the Operation:
If Z = 1:
Shift R2 left by 2 bits: This operation multiplies the value in R2 by 4.
Add the shifted value of R2 to the value in R1.
Store the result in R0.

Step 3. Update the Flags:
The S suffix means the flags will be updated based on the result of the addition:

- N (Negative): Set if the result is negative.
- Z (Zero): Set if the result is zero.
- C (Carry): Set if there was a carry out of the most significant bit during the addition.
- V (Overflow): Set if there was an overflow (e.g., adding two positive numbers results in a negative number or vice versa).

Solution:

1. **Shift R2 Left by 2 Bits:**
   - R2 = 0x00000003 (binary: 0000 0000 0000 0000 0000 0000 0000 0011)
   - R2 LSL #2 = 0x0000000C (binary: 0000 0000 0000 0000 0000 0000 0000 1100) - left shifted by 2 bits.
2. **Add R1 and the Shifted R2:**
   R1 + (R2 LSL #2) = 0x00000005 + 0x0000000C = 0x00000011
3. **Store the Result in R0:**
   R0 = 0x00000011
4. **Update Flags:**
   - N = 0 (The result is positive, so the N flag is 0.)
   - Z = 0 (The result is not zero, so the Z flag is 0.)
   - C = 0 (No carry occurred in this simple addition.)
   - V = 0 (No overflow occurred.)

Therefore, R0 will hold the value 0x00000011. Flags will be updated based on the result.

6.3 Introduction of Flag-setting Suffix {S}

In ARMv7, many instructions can be conditionally executed based on the flag-setting suffix {S}, which refers to the updates of condition flags (N, Z, C, V). if {S} present, instructions need to update all four flags based on the operation final result. Otherwise, flags will not be affected.

Example 1: ADD instruction **with the suffix S**, flags changes according to the result.

Instruction:
ADDS R0, R1, R2

Assume the flags before this instruction is: N=0, Z=0, C=0, V=0;

- R1 = 0x80000000 (which is -2147483648 in signed 32-bit integer)
- R2 = 0x80000000 (which is -2147483648 in signed 32-bit integer)

Operation: R0 = R1 + R2, 0x80000000 + 0x80000000 = 0x100000000

Since R0 is a 32-bit register, only the Lower 32 bits are stored, so R0=0x00000000
Lower 32 bits = The 32 bits that include the LSB and extend up to the 32nd bit position.
The "upper" bit (the 33rd bit) is not stored in the 32-bit register but is considered as an overflow, which affects the carry flag (C).

Flags Affected

- N (Negative): Since the result is 0, N = 0.
- Z (Zero): Since the result is 0, Z = 1.
- C (Carry): The result overflowed the 32-bit limit (a carry out of the MSB occurred), so C = 1.
- V (Overflow): The addition of two large negative numbers resulted in a non-negative number, so V = 1.

Final Flags after ADDS: N = 0, Z = 1, C = 1, V = 1

Example 2: ADD instruction **without the suffix S**, flags are not affected by the result.

Instruction: **ADD R0, R1, R2**

Assume the flags before this instruction is: N=0, Z=0, C=0, V=0;

- R1 = 0x80000000 (which is -2147483648 in signed 32-bit integer)

- R2 = 0x80000000 (which is -2147483648 in signed 32-bit integer)

Operation: R0 = R1 + R2, 0x80000000 + 0x80000000 = 0x100000000
Final Flags after ADD: N = 0, Z = 0, C = 0, V = 0. Since there is no 'S' suffix, the flags are not affected.

6.4 Introduction of the Argument2 of ALU - <Oprnd2>

<Oprnd2> refers to an operand is the part of the instruction that references data or a peripheral device.

Operand	Type	Mnemonic
Operand 2 <Oprnd2>	Immediate value	`#32bit_Imm`
	Logical shift left	`Rm LSL #5bit_Imm`
	Logical shift right	`Rm LSR #5bit_Imm`
	Arithmetic shift right	`Rm ASR #5bit_Imm`
	Rotate right	`Rm ROR #5bit_Imm`
	Register	`Rm`
	Logical shift left	`Rm LSL Rs`
	Logical shift right	`Rm LSR Rs`
	Arithmetic shift right	`Rm ASR Rs`
	Rotate right	`Rm ROR Rs`
	Rotate right extended	`Rm RRX`

Fig 6.1 Operand Types

Examples:

1) ADD R0, R1, #1

- R0: Destination register where the result will be stored.
- R1: Source register 1.
- #1: Immediate value (1 in this case).

This instruction adds the immediate value 1 to the contents of R1 and stores the result in R0.Operand 2 is an immediate value (#1).

2) ADD R0, R3, R1, LSL #2

- R0: Destination register where the result will be stored.
- R3: Source register 3
- R1, LSL #2: Operand 2, where R1 is shifted left logically (LSL) by 2 positions.

This instruction performs a logical shift left (LSL) on the value in R1 by 2 bits, effectively multiplying it by 4. The shifted value is then added to the contents of R3, and the result is stored in R0. Operand 2 is R1 after it has been shifted left by 2 positions (LSL #2).

3) ADD R0, R1, R1

- R0: Destination register where the result will be stored.
- R1: Source register 1
- R1: Operand 2

This instruction simply adds the value in R1 to the value in R1 and stores the result in R0. Operand 2 is the contents of R1.

4) ADD R0, R3, R1, LSL R2

- R0: Destination register where the result will be stored.
- R3: Source register 3
- R1, LSL #2: Operand 2, where R1 is logically shifted left by the value in R2.

This instruction performs a logical shift left (LSL) on the value in R1 by by a number of positions specified by the value in R2. The result of this shift operation is then added to the contents of R3, and the final result is stored in R0. Operand 2 is R1 after it has been shifted left by a value contained in R2 (LSL R2).

6.5 Addition with carry instruction: ADC{cond}{S} Rd, Rn, <Oprnd2>

Given two 32-bit registers, and the sum overflows the capacity of a 32-bit register (e.g., a result greater than 2^32 − 1). In this case, the carry flag will be set. Then, when you perform the ADC on the next part of the multi-precision number (the upper part of the number), the carry is added in, ensuring the full value is correctly summed. For example:

ADC R0, R1, R2

R0 = R1+R2+ Carry flag. It performs an addition between the values in R1 and R2, and it also includes the value of the carry flag (C flag) from the status register. The carry flag comes from a previous operation, typically from an addition that overflowed beyond the bit size of the register

R1 = 0xFFFFFFFF (largest possible 32-bit number), R2 = 1, carry flag = 1.

Result: R0 = R1 + R2 + C = 0xFFFFFFFF + 1 + 1 = 0x00000001 (32-bit overflow handled by adding the carry).

6.6 Data move instruction: MOV{cond}{S} Rd, <Oprnd2>

The MOVE instruction essentially copies data from one place to another, typically moving data between registers. MOVE is critical for controlling program flow, managing exceptions, and updating the status registers that govern how the processor reacts to different conditions.

Operation		Assembly syntax
Move	Move	`MOV{cond}{S} Rd, <Oprnd2>`
	Move NOT	`MVN{cond}{S} Rd, <Oprnd2>`
	Move SPSR to register	`MRS{cond} Rd, SPSR`
	Move CPSR to register	`MRS{cond} Rd, CPSR`
	Move register to SPSR	`MSR{cond} SPSR{field}, Rm`
	Move register to CPSR	`MSR{cond} CPSR{field}, Rm`
	Move immediate to SPSR flags	`MSR{cond} SPSR_f, #32bit_Imm`
	Move immediate to CPSR flags	`MSR{cond} CPSR_f, #32bit_Imm`

Fig 6.2 Move Operation

6.7 MOV with Status Registers

MOV instruction also interact with status registers: CPSR (Current Program Status Register) and SPSR (Saved Program Status Register).

6.7.1 Syntax: MRS{cond} Rd, SPSR

- The value of the SPSR (Saved Program Status Register) is moved to the general-purpose register Rd.
- SPSR contains information about the processor's state and is used when returning from an exception.
- Use Case: When you want to save or inspect the SPSR.

6.7.2 Syntax: MRS{cond} Rd, CPSR

- This instruction moves the value of the CPSR (Current Program Status Register) into the general-purpose register Rd.

- CPSR holds the current state of the program, including the mode and Flags (N, Z, C, V).
- Use Case: When you want to check or manipulate the current status flags during program execution.

6.7.3 Syntax: MSR{cond} SPSR{field}, Rm

- This moves the value in the general-purpose register Rm into the SPSR. The field indicates specific parts of the SPSR that are being updated.
- Use Case: This is used in exception handling when you need to save the current status (from CPSR) in the SPSR.

6.7.4 Syntax: MSR{cond} CPSR{field}, Rm

- This moves the value in register Rm to the CPSR, which can be used to directly manipulate the status register.
- Use Case: This might be used for updating flags or other fields of the CPSR directly.

6.8 Subtraction Instructions: SUB{S} Rd, Rn, Operand2

A series of SUB instructions performs subtraction but with slight variations in behavior and intended use.

6.8.1 Syntax: SUB{S} Rd, Rn, Operand2

- Subtracts the second operand from the first and stores the result in the destination register.
- The condition flags can be updated based on the result if the "S" suffix is used (i.e., SUBS). C (Carry): Set to 1 if the subtraction did not result in a borrow (i.e., if Rn >= Operand2)

  Example:

```
MOV R0, #10          // Load 10 into R0
MOV R1, #3           // Load 3 into R1
SUBS R2, R0, R1      // R2 = R0 - R1 = 10 - 3 = 7.
                     // The result of 10-3 is positive (7), so no borrow is
                        needed.
                     // Then we set Carry to 1.
```

```
MOV R0, #5          // Load 5 into R0
MOV R1, #10         // Load 10 into R1
SUBS R2, R0, R1     // R2 = R0 - R1 = 5 – 10 = -5
```

The result of 5-10 is negative (-5), so a borrow is needed. Then we set Carry to 0.

6.8.2 Syntax: SBC{S} Rd, Rn, Operand2

Similar with ADC, the SBC instruction performs subtraction while considering the current state of the carry flag. The carry flag (C) is subtracted from the result.

The condition flags can be updated based on the result if the "S" suffix is used (i.e., SUBS).

Calculation: Rd = Rn – Operand2 - (1 – C)

- C = 1: If the carry flag is set (C = 1), it means no borrow occurred in the previous subtraction, so you subtract 0 from the result.
- C = 0: If the carry flag is cleared (C = 0), it means a borrow did occur in the previous subtraction, so you subtract 1 to account for the borrow.

6.8.3 Syntax: RSB{S} Rd, Rn, Operand2

Reverse Subtract (RS), subtracting the first operand from the second operand in reverse order. Therefore, Rd = Operand2 – Rn

Use Case: Negating a value, subtract the original value from zero

Example: RSB R0, R1, R2. R0=R2-R1

6.9 Additional Instructions

6.9.1 Syntax: CMP Rn, Operand2

Comparing two values by performing a subtraction but without storing the result. Example: CMP R1, R2

- Apply SUB on R1 and R2 (R1-R2)
- Change Flags according to the result
- Discard the result

6.9.2 Syntax: CMN Rn, Operand2

Perform a comparison by adding the negative (2's complement) of the second operand to the first operand. Then updates the condition flags (N, Z, C, V) based on the result of ADD, but it doesn't store the result.

Use Case: Comparison in conditional branching

Example: CMN R1, R2

- Apply ADD on R1 and R2
- Change Flags according to the result
- Discard the result

6.9.3 Syntax: MUL{S} Rd, Rn, Rm

Multiplication of two registers and store the result in a destination register, represented by Rd = Rn * Rm. MUL performs 32-bit multiplication. If the result of the multiplication exceeds 32 bits, the upper bits will be truncated.

6.9.4 Syntax: MLA{S} Rd, Rn, Rm, Ra

MLA (Multiply-Accumulate) performs a multiplication of two registers and adds the result to a third register in a single instruction.

Operation: Rd = (Rn * Rm) + Ra

MLAS need to update flags (Z, N) based on results.

Use Case: both multiplication and accumulation, e.g., matrix operations

Chapter 7. Data in Memory and the Load-Store Architecture

7.1 The Role of Memory

In any modern computer system, the Central Processing Unit (CPU) is the brain, performing calculations and executing instructions. However, the CPU does not have enough storage to hold all the programs and data it needs to work on. For this, it relies on the main memory, also known as Random Access Memory (RAM).

When you run a program, whether it's your operating system, a web browser, or a game, its instructions and the data it operates on are loaded into RAM. This is why RAM is often called the primary or main memory. It serves as a temporary workspace for the computer. When you turn off the computer, the contents of RAM are lost, which is why it is referred to as volatile memory.

The CPU's Arithmetic Logic Unit (ALU), which performs calculations like addition and subtraction, cannot access this main memory directly. Instead, it interacts with a small, extremely fast set of storage locations inside the CPU called registers. Data must first be brought from the main memory into a register before the ALU can work on it. This fundamental design is known as a Load-Store Architecture.

- **Load**: An operation that copies data *from* memory *to* a register.
- **Store**: An operation that copies data *from* a register *to* memory.

This chapter will explore how data is organized in memory, how the load and store operations work in the ARMv7 architecture, and the different ways the CPU can specify memory locations.

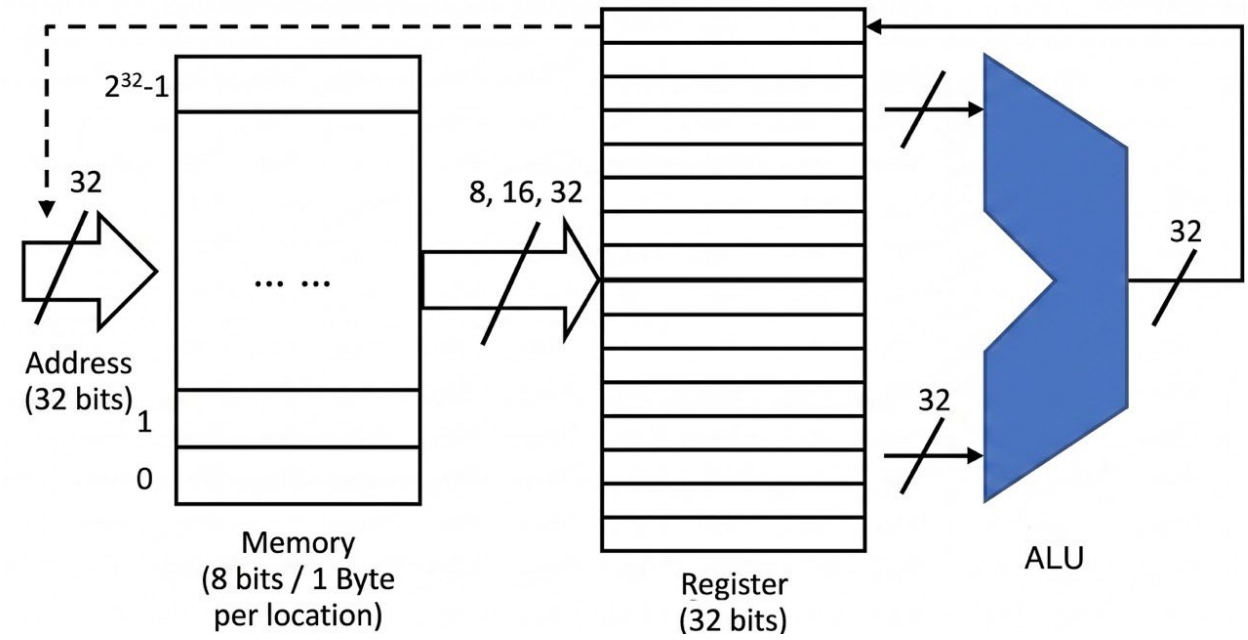


Fig 7.1: Data Transfer Interface

7.2 Memory Organization

To work with memory, we must first understand how it is organized. Main memory can be visualized as a very long sequence of storage cells.

7.2.1 Byte-Addressable Memory

The fundamental unit of memory is the byte, which is a group of 8 bits. Every single byte in the main memory has a unique numerical identifier called its address. This is similar to how every house on a street has a unique address. When the CPU needs to access a specific piece of data, it uses its address. This is why we call it *byte-addressable memory*.

In a 32-bit system, memory addresses are 32 bits long. This means there are 2^32 possible unique addresses, allowing the system to address up to 4 gigabytes (GB) of memory.

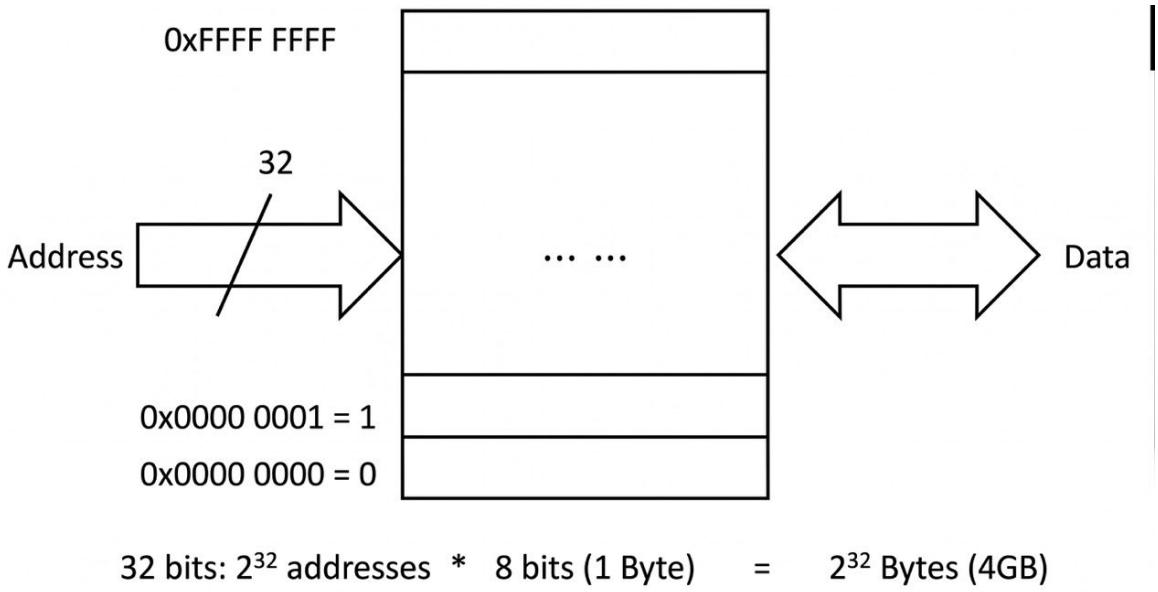


Fig 7.2: Memory Address and Storage Capacity

7.2.2 Pointers and Memory Addresses

If you have programmed in languages like C or C++, you have likely used pointers. A pointer is simply a variable that holds a memory address. When you "dereference" a pointer, you are telling the computer to go to the memory address stored in the pointer and retrieve the value at that location. This is a high-level abstraction of the load and store operations we will discuss.

```
// C language example
int main() {
    char c = 'A'; // char is 1 byte, stored at some memory address
    char *p = &c; // The pointer 'p' now holds the address of 'c'

    // *p dereferences the pointer to get the value 'A'
    printf("%c\n", *p);

    return 0;
}
```

7.3 Endianness: How Multi-Byte Data is Stored

When we need to store a piece of data that is larger than one byte (e.g., a 16-bit half-word or a 32-bit word), we must use multiple consecutive memory locations. For example, a 32-

bit word requires four bytes. This raises a question: in what order should the bytes of the word be stored in memory?

The rule that dictates this ordering is called endianness. There are two primary systems of endianness.

7.3.1 Little-Endian

In a Little-Endian system, the "little end" (the least significant byte) of the data is stored at the lowest memory address. The remaining bytes are stored in order at the subsequent higher addresses. The ARM architecture, used in the Raspberry Pi, is typically Little-Endian.

7.3.2 Big-Endian

In a Big-Endian system, the "big end" (the most significant byte) of the data is stored at the lowest memory address. This is often considered more intuitive for humans reading hexadecimal dumps, as it matches the left-to-right reading order.

Example: Storing a 32-bit Word

Let's say we want to store the 32-bit hexadecimal value 0xABCDEF90 at the starting memory address 0x1000. This value consists of four bytes: AB, CD, EF, and 90.

- Most Significant Byte (MSB): 0xAB
- Least Significant Byte (LSB): 0x90

The following table shows how this value would be stored in both Little-Endian and Big-Endian systems.

Address	Little-Endian	Big-Endian
0x1003	0xAB	0x90
0x1002	0xCD	0xEF
0x1001	0xEF	0xCD
0x1000	0x90	0xAB

Table 7. 1 Value Stored in Little-Endian and Big-Endian

Understanding endianness is critical because when you transfer data between systems with different endianness, you might need to convert the byte order to interpret the data correctly.

7.4 ARMv7 Store Instructions

Store instructions copy data from a register to memory. The ARMv7 instruction set provides different instructions based on the size of the data being stored: a byte (8 bits), a half-word (16 bits), or a word (32 bits).

The general syntax is INSTRUCTION{cond} Rd, [Rn], where:

- Rd is the source register containing the data to be stored.
- Rn is the register containing the memory address to store the data at. The square brackets [] indicate that we are using the value in Rn as an address (i.e., dereferencing it)

7.4.1 STRB: Store Byte

The STRB instruction stores the least significant byte (bits [7:0]) of the source register Rd into the single memory byte at the address specified by Rn.

Example

Assume R0 = 0xABCDEF90 and R1 = 0xF000.

STRB R0, [R1]

This instruction will take the least significant byte of R0, which is 0x90, and store it at memory address 0xF000.

7.4.2 STRH: Store Half-word

The STRH instruction stores the least significant half-word (bits [15:0]) of Rd into two consecutive bytes in memory, starting at the address in Rn. The exact byte ordering depends on the system's endianness.

Example

Assume R0 = 0xABCDEF90 and R1 = 0xF000. The half-word to be stored is 0xEF90.

STRH R0, [R1]

- **Little-Endian:** The least significant byte, 0x90, is stored at the lower address 0xF000. The byte 0xEF is stored at the higher address 0xF001.
- **Big-Endian:** The most significant byte, 0xEF, is stored at the lower address 0xF000. The byte 0x90 is stored at the higher address 0xF001.

7.4.3 STR: Store Word

The STR instruction stores the full 32-bit word from Rd into four consecutive bytes in memory, starting at the address in Rn. Again, endianness determines the byte order.

Example

Assume R0 = 0xABCDEF90 and R1 = 0xF000.

```
STR R0, [R1]
```

- **Little-Endian:** Address 0xF000 gets 0x90; Address 0xF001 gets 0xEF; Address 0xF002 gets 0xCD; Address 0xF003 gets 0xAB
- **Big-Endian:** Address 0xF000 gets 0xAB; Address 0xF001 gets 0xCD; Address 0xF002 gets 0xEF; Address 0xF003 gets 0x90.

7.5 ARMv7 Load Instructions

Load instructions perform the opposite operation: they copy data from memory into a register. ARM provides instructions for loading bytes, half-words, and words. A key difference from store instructions is that when loading a value smaller than 32 bits (a byte or half-word), the CPU must decide what to do with the remaining upper bits in the 32-bit destination register. This leads to two variants for smaller loads: *unsigned* and *signed*.

7.5.1 LDRB: Load Unsigned Byte

The LDRB instruction loads a single byte from the memory address in Rn into the least significant byte of Rd (bits [7:0]). It then fills the upper 24 bits of Rd with zeros. This is an *unsigned* load.

Example

Assume memory at address 0xF000 contains the byte 0x9A (binary 10011010).

LDRB R0, [R1] // R1 contains 0xF000

After execution, R0 will be 0x0000009A.

7.5.2 LDRSB: Load Signed Byte

The LDRSB instruction also loads a single byte from memory. However, it treats the loaded byte as a signed 8-bit number (in two's complement). It copies the sign bit (bit 7) of the byte into the upper 24 bits of the destination register Rd. This is called *sign extension*.

Example

Assume memory at address 0xF000 contains the byte 0x9A. The sign bit of 0x9A (binary 10011010) is 1, indicating a negative number.

```
LDRSB R0, [R1]  // R1 contains 0xF000
```

After execution, R0 will be 0xFFFFFF9A. The sign bit has been extended across the entire register. If the loaded byte had been 0x1D (binary 00011101), the sign bit would be 0, and the result would be 0x0000001D.

7.5.3 LDRH and LDRSH: Load Half-word

These instructions load a 16-bit half-word from memory. The byte ordering is determined by endianness.

- LDRH: Performs an *unsigned* load, filling the upper 16 bits of Rd with zeros.
- LDRSH: Performs a *signed* load, extending the sign bit (bit 15) of the half-word into the upper 16 bits of Rd.

7.5.4 LDR: Load Word

The LDR instruction loads a full 32-bit word from memory into a register. Since the data size matches the register size, there is no need for sign extension or zero-padding. The byte ordering is determined by endianness.

7.6 Addressing Modes

ARM provides several flexible ways to calculate the memory address for load and store operations. These are called *addressing modes*.

7.6.1 Register Direct Addressing

This is the mode we have used so far: [Rn]. The address is simply the value contained in register Rn.

7.6.2 Offset Addressing

This mode calculates the address by adding an offset to a base register. The base register itself is not changed.

- Immediate Offset: [Rn, #immediate]. The address is Rn + immediate.

```
STR R0, [R1, #4]        // Address = value in R1 + 4
```

- Register Offset: [Rn, Rm]. The address is Rn + Rm.

```
LDR R0, [R1, R2]        // Address = value in R1 + value in R2
```

7.6.3 Post-Indexed Addressing

This mode is very useful for loops that process sequential data in memory. It uses the value in Rn as the address for the load/store operation, and then updates Rn by adding an offset. The base register Rn is modified. The syntax is distinguished by placing the offset outside the brackets.

- Post-indexed with Immediate: [Rn], #immediate.

```
LDR R0, [R1], #4
```

This is equivalent to two separate operations:

```
LDR R0, [R1]            // Load from address in R1
ADD R1, R1, #4          // Update R1
```

- Post-indexed with Register: [Rn], Rm.

```
LDR R0, [R1], R2
```

This is equivalent to:

```
LDR R0, [R1]            //Load from address in R1
ADD R1, R1, R2          //Update R1 with value from R2
```

Chapter 8. Branch Instructions, Procedure Calling Mechanism, and Assembly Implementation

Branch is an instruction that allows us to change the flow of program execution. Without Branch, program is executed in a sequential flow, line by line. There are two types of branch:

- Branch always (unconditionally)
    - Syntax Example: B, BL, BX
- Branch conditionally
    - Syntax Example: B[cond]

8.1 Link Register and Program Counter Register

Before we learn the Branch series instructions, let's look at two essential registers for Branch.

8.1.1 Register LR (R14): Link Register

LR holds the address to return when a subroutine call completes.

8.1.2 Register PC (R15): Program Counter

CPU employs a general register R15 store the address of the next instruction to be executed. The register R15 is named as Program Counter (PC), it is also called instruction counter, instruction pointer, and instruction address register (IAR).

It is a digital counter that is needed to execute tasks quickly and track the current execution point. All the instructions and data present in memory have a special address. As each instruction is processed, the program counter is updated to the address of the next instruction to be fetched.

PC stores the address of the next instruction. After executing one instruction, PC increment by 4 to point to the next instruction (if no branch). The reason for the increase by 4 is that each instruction is 32 bits (4 bytes).

8.2 Syntax of Branch Instructions

Following the same two types of branch instruction we introduced, here are the syntaxes of branch instructions.

8.2.1 Syntax: B label

jump to the position based on the *label* (i.e., the instruction right after the label).
Label is the address of the line of code below it. Essentially, the instruction copy the address of the *label* to PC (R15), so the instruction executed next will be the one after the label

8.2.2 Syntax: BL label

When BL is executed, two tasks will be completed:
Task 1 – Take the branch: it causes the program to jump to the target address specified by the label.
Task 2 – Stores Return Address: it saves the address of the next instruction (i.e., the one after BL) in the Link Register (LR). This saved return address allows the program to return to the right place after the subroutine completes. The BL instruction is used to call a subroutine or function. In fact, Copy PC to LR and then copy the address of label to PC.

Unlike instruction 'B', the instruction 'BL' preserve the content of PC with LR so that the program can branch back. BL is commonly used at the start of a function call.

8.2.3 Syntax: BX LR

The the BX LR instruction is typically used to return from a subroutine (Copy LR to PC).

When you execute BX LR, the program counter (PC) is set to the value in the Link Register (LR). The LR holds the return address from the subroutine, allowing the program to continue execution from where it left off.

Exchange Feature: The "Exchange" part of BX also refers to the ability to switch between ARM and Thumb instruction sets based on the least significant bit of the target address. However, in most typical use cases like BX LR, it's just used for returning from a subroutine.

Example:

```
MOV R0, #3          // Load immediate value 3 into R0
BL my_subroutine    // Branch to the subroutine 'my_subroutine' and
```

```
                        // save return address in LR
MOV R1, R0              // Move the value in R0 to R1 (executed after returning
                        // from the subroutine)
my_subroutine:
ADD R0, R0, #7          // Add 7 to the value in R0 (R0 becomes 10)
BX LR                   // Branch back to the address stored in LR
                        // (return from subroutine)
```

Question: What's the value of R1 after executed the code above?
Answer: R1 = 10
The register LR saved the return address (which is the next instruction of BL my_subroutine). Then the execution returns to the MOV R1, R0 after the execution of BX LR at the end.

In combination, BL and BX LR work together to manage the control flow between calling functions and returning, similar to the function calls and returns in high-level languages like C.

8.2.4 Syntax: B[cond] label

The B{cond} instruction in ARM assembly is used for conditional branching.

It allows the program to branch to a specified label or address if a specific condition is met, based on the values in condition flags (Negative, Zero, Carry, Overflow) set by previous instructions.

Branch	Conditional	-
	• if Z set	`BEQ label`
	• if Z clear	`BNE label`
	• if C set	`BCS label`
	• if C clear	`BCC label`
	• if N set	`BMI label`
	• if N clear	`BPL label`
	• if V set	`BVS label`
	• if V clear	`BVC label`
	• if C set and Z clear	`BHI label`
	• if C clear and Z set	`BLS label`
	• if ((N set and V set) or (N clear and V clear))	`BGE label`
	• if ((N set and V clear) or if (N clear and V set))	`BLT label`
	• if (Z clear and ((N or V set) or (N or V clear)))	`BGT label`
	• if (Z set or ((N set and V clear) or (N clear and V set)))	`BLE label`

Fig 8.1 Branch Conditions

Field type	Suffix	Description	Condition
Condition {cond}	EQ	Equal	Z set
	NE	Not equal	Z clear
	CS	Unsigned higher, or same	C set
	CC	Unsigned lower	C clear
	MI	Negative	N set
	PL	Positive, or zero	N clear
	VS	Overflow	V set
	VC	No overflow	V clear
	HI	Unsigned higher	C set, Z clear
	LS	Unsigned lower, or same	C clear, Z set
	GE	Greater, or equal	N=V (N and V set or N and V clear)
	LT	Less than	N<>V (N set and V clear) or (N clear and V set)

Fig 8.2 Branch Conditions Code

8.2.4.1 Branch is NOT Taken

```
MOV R0, #5          // Load the immediate value 5 into R0
MOV R1, #10         // Load the immediate value 10 into R1
CMP R0, R1          // Compare R0, R1 (sets condition flags). R0 is less than R1,
                    // Zero flag (Z) is cleared, and the Negative flag (N) is set.
BGT greater_label   // Branch if Greater Than (BGT). BGT branches if N == 0
                    // and Z == 0, which means if the R0 > R1. In this example,
                    // R0 is not greater than R1, this branch is not taken, and the
                    // program continues with the next instruction.
MOV R2, #0                  // Since R0 is not greater than R1, the program
                    continues
                    // here after the False of BGT condition check.
B end_label         // Jump to the end_label, proceed to the end. Consider the
                    // end_label as a placeholder to mark the end of the
                    // execution flow. Prevent unwanted execution.
greater_label:
    MOV R2, #1      // This instruction will be executed only if R0 > R1
    B end_label     // Jump to the end_label.
end_label:
```

```
        BX LR
```

8.2.4.2 Branch is Taken

```
MOV R0, #10          // Load the immediate value 10 into R0
MOV R1, #5           // Load the immediate value 5 into R1
CMP R0, R1           // Compare R0, R1 (sets condition flags). R0 is less than R1,
                      // Zero flag (Z) is cleared, and the Negative flag (N) is set.
BGT greater_label    // Branch if Greater Than (BGT). BGT branches if N == 0 and
                      // Z == 0, which means if the R0 > R1. In this example,
                      // R0 is greater than R1, this branch is taken, and the
                      // program jump to the subroutine called “greater_label”.
MOV R2, #0
B end_label
greater_label:
        MOV R2, #1   // This instruction will be executed only if R0 > R1
        B end_label  // Jump to the end_label.
end_label:
        BX LR
```

8.3 Procedure Calling Mechanism

A procedure (or function) is a block of reusable code designed to perform a specific task. In ARMv7 assembly language, procedures help in breaking down complex programs into smaller, more manageable pieces.

For example, procedure calling in ARMv7:

```
MOV R0, #5                  // Load 5 in R0
MOV R1, #10                 // Load 10 in R1
BL add_numbers              // Call the function 'add_numbers'
add_numbers:
        ADD R0, R0, R1      // Add the values in R0 and R1
        BX LR               // Return to the caller
```

The BL (Branch with Link) instruction is used to call a procedure. BL saves the return address (address of the next instruction) in the LR (Link Register, R14). After the procedure completes, BX LR (or MOV PC, LR) is used to return to the caller.

8.3.1 Role of Procedures

- Modular Code Design: Procedures allow a large program to be broken into smaller, reusable functions.
- Code Reusability: Functions can be reused in different parts of the program, avoiding code repetition.
- Separation of Concerns: Makes it easier to understand, test, and debug each component separately.

8.3.2 Role of the Link Register (LR)

- The LR is crucial for procedure calls. It holds the return address so that the program can continue executing after the procedure completes.
- LR is automatically set by the BL instruction.
- The called procedure uses LR to return control back to the caller using BX LR (or MOV PC, LR).

8.3.3 Returning from a Procedure:

To return from a procedure, ARMv7 typically uses one of the following instructions:

- BX LR: Branch to the address stored in LR.
- MOV PC, LR: Move the contents of LR to the PC (Program Counter) to continue from where it left off.

8.4 Flow of Control

Caller Side: The caller sets up arguments in the appropriate registers (R0 to R3 for the first four arguments). Calls the procedure with BL.

Callee Side: The callee executes the procedure. The return is handled using BX LR or MOV PC, LR.

- **Step 1**. Caller Prepares. The caller stores parameters in specific registers (typically R0-R3).
- **Step 2.** Branch with Link (BL). The **BL** instruction is used to call the procedure.The current value of the Program Counter (PC) is saved in **LR** for later return.
- **Step 3.** Procedure Execution. The callee performs its designated tasks.
- **Step 4.** Return (**BX LR** or **MOV PC, LR**). The callee uses the **BX LR** instruction to return to the caller. Control is transferred back to the caller by jumping to the address stored in **LR**.

8.5 Loop with Conditional Branch Instruction

We use a classic question CountOnes32 to explain the look with conditional Branch instruction.

Question: Write the assembly code based on the following function

extern uint32_t countOnes32 (uint32_t x);

This function will return the number of bits 1 in x. e.g., x = 0b 0000 0000 0000 0000 0000 1111 0000 0000, return 4.

8.5.1 Solution A: Shifting

Shifting bits in x to C flag one by one and examine each of them

- Step 1: Logic Shift argument x to Left (or Right) by one bit (essentially move MSB (or LSB) bit to C flag)
- Step 2: Examine C flag
    - If C set (a bit 1), increment the result by 1
    - If C clear (a bit 0), no action
- Step 3: branch back to step 1 until only bit 0 left in argument (arg = 0, Z flag set)

8.5.1.1 Assembly code

countOnes32:

```
    MOV R1, R0          // Copies the input value (x) from R0 to R1. R1 will be used
                        // for masking each bit without modifying the original value.
    MOV R0, #0          // Initializes R0 to 0, stores the count of bits set to 1.
Start:
    MOVS R1, R1, LSR #1 // Shifts R1 right by 1 bit and updates the condition flags
                        // (S suffix means update flags). The least significant bit
                        // (LSB) that is shifted out is moved to the carry flag (C).
    ADDCS R0, R0, #1    // Adds 1 to R0 if the carry flag is set (CS stands for "Carry
                        // Set"). If the bit shifted out (the LSB of R1) is 1, the carry
                        // flag is set, and R0 (the bit count) is incremented.
```

BNE Start

BX LR

Branch back to the Start label. The instruction that directly affects whether BNE will branch depends on the most recent instruction that sets condition flags. The previous instruction MOVS R1, R1, LSR #1 is the most recent instruction (not ADDCS!) that affects the condition flags, specifically the Zero flag (Z).

After MOVS R1, R1, LSR #1,

if R1 becomes zero (i.e., all bits have been shifted out), the Zero flag (Z) will be set, and BNE will not branch. Then proceed to BX LR.

If R1 is not zero after shifting, the Zero flag (Z) will be cleared, and BNE will branch to Start repeatedly.

8.5.1.2 Example of real values plugged in

Step 1. Initialization

R0 = 0b00000000000000000000111100001001

R1 = R0 → R1 = 0b00000000000000000000111100001001

R0 = #0 → R0 = 0 (This will be used to store the count of 1s)

Step 2. Iterations

1. **Iteration 1**

R1 = 0b00000000000000000000111100001001

LSR R1 → R1 = 0b00000000000000000000011110000100

The shifted-out bit is 1 (carry set), so executing ADDCS R0, R0, #1: R0 = R0 + 1 → R0 = 1. Thus, R0 represents how many times we met the target '1'

2. **Iteration 2**

R1 = 0b00000000000000000000011110000100

LSR R1 → R1 = 0b00000000000000000000001111000010

The shifted-out bit is 0 (carry not set), so R0 remains 1, no change.

3. **Iteration 3**

R1 = 0b00000000000000000000001111000010

LSR R1 → R1 = 0b00000000000000000000000111100001

The shifted-out bit is 0 (carry not set), so R0 remains 1, no change.

4. **Iteration 4**

R1 = 0b00000000000000000000000111100001

LSR R1 → R1 = 0b00000000000000000000000011110000

The shifted-out bit is 1 (carry set), so R0 = R0 + 1 → R0 = 2. Now we've met the target '1' two times.

5. **~ 12 Iterations 5 ~ 12**

Continuing to shift R1 to the right:

Iteration 5: R1 = 0b00000000000000000000000011110000 → LSR → Update R1 and R0 → … … → Iteration 12: R1 = 0b00000000000000000000000000000001, LSR R1 → R1 = 0b00000000000000000000000000000000, the shifted-out bit is 1 (carry set), so R0 = R0 + 1 → R0 = 6.

R1 is now 0, so the loop ends

Step 3. Final Result

R0 = 6. The original value of R0 (0b00000000000000000000111100001001) had 6 bits set to 1, and this count is stored in R0 at the end of the code execution.

8.5.2 Solution B: Masking

We use the method of masking to examine each bit in the argument.

Step 1: Initialize Mask as #1 (examine the 32 bits from right to left)

Step 2: arg & Mask (examine one bit)

- Z clear: the bit is 1, increment the result by 1
- Z set: the bit is 0, no action

Step 3: Shift the Mask to left by 1 bit (for the examination of the next bit)

- Z clear: Mask is not zero, branch back to step 1
- Z set: Mask is zero, all the 32 bits are examined, return.

8.5.2.1 Assembly Code

countOnes32:

```
MOV R1, R0          // Copies the input value (x) from R0 to R1. R1 will be used for
                    // masking each bit without modifying the original value.
MOV R0, #0          // Initializes R0 to 0, stores the count of bits set to 1.
MOV R2, #1          // R2 act as a bitmask to isolate each bit in R1 by shifting left.
```

Start:

```
ANDS R3, R1, R2     // Bitwise AND operation between R1 and R2, storing the result in
                    // R3. Any better options to replace ANDS?  - TST R1, R2. It only
                    // updates condition flags based on the result of the AND operation.
                    // Just like ANDS, but does not store the result. Using TST is more
                    // concise and efficient because it directly performs the operation
                    // needed for flag checking and does not require an additional
                    // register (R3) for storing the result.
ADDNE R0, R0, #1    // Adds 1 to R0 if the result of the previous ANDS operation
                    // was not equal to zero (NE means "Not Equal").This means
                    // that if the bit in R1 matched the bitmask in R2, increment
                    // the count of 1 bits.
MOVS R2, R2, LSL #1 // Logically shifts R2 left by 1 bit, effectively moving the
```

```
                    // bitmask to the next bit position.
BNE Start           // Branches back to the Start label if R1 is not equal to 0
                    // (NE stands for "Not Equal"). This means the loop continues
                    // until all bits in R1 are processed (i.e., until R1 becomes 0).
BX LR
```

8.5.2.2 Example of real values plugged in

Step 1. Initialization:

- R0 = 0b00000000000000000000111100001001 (Initial input value, x)
- R1 = R0 → R1 = 0b00000000000000000000111100001001
- R0 = 0 (Initialize bit count)
- R2 = 1 (Bitmask to check each bit from LSB to MSB)

Step 2. Iterations

1. **Iteration 1 (R2 = 0b00000000000000000000000000000001):**
   - TST R1, R2 → R1 & R2 = 0b00000000000000000000111100001001 & 0b00000000000000000000000000000001 = 1
   - Result is non-zero (1), which sets the NE (Not Equal) condition.
   - ADDNE R0, R0, #1 → R0 = 0 + 1 = 1 (Increment count)
   - MOVS R2, R2, LSL #1 → R2 = 0b00000000000000000000000000000010
   - BNE Start (since R2 is not zero, continue)
2. **Iteration 2 (R2 = 0b00000000000000000000000000000010):**
   - TST R1, R2 → R1 & R2 = 0b00000000000000000000111100001001 & 0b00000000000000000000000000000010 = 0
     - Result is zero, NE condition is not set.
   - ADDNE R0, R0, #1 (Not executed since NE is not set)
   - MOVS R2, R2, LSL #1 → R2 = 0b00000000000000000000000000000100
   - BNE Start (continue)
3. **Iteration 3 (R2 = 0b00000000000000000000000000000100):**
   - TST R1, R2 → R1 & R2 = 0b00000000000000000000111100001001 & 0b00000000000000000000000000000100 = 0
     - Result is zero, NE condition is not set.
   - ADDNE R0, R0, #1 (Not executed)
   - MOVS R2, R2, LSL #1 → R2 = 0b00000000000000000000000000001000
   - BNE Start (continue)

4. **Iteration 4 (R2 = 0b00000000000000000000000000001000):**
    - TST R1, R2 → R1 & R2 = 0b00000000000000000000111100001001 & 0b00000000000000000000000000001000 = 0b00001000
        - Result is non-zero (8), which sets the NE condition.
    - ADDNE R0, R0, #1 → R0 = 1 + 1 = 2 (Increment count)
    - MOVS R2, R2, LSL #1 → R2 = 0b00000000000000000000000000010000
    - BNE Start (continue). This process continues for each bit in the 32-bit value (R1). Each iteration will test the next bit by shifting R2 to the left. If a bit in R1 is 1, the count (R0) is incremented.

Step 3. Summary of Count:

For the given value 0b00000000000000000000111100001001:

- Bits set to 1: 1st, 4th, 12th, 13th, 14th, 15th (from LSB) → Total 6
- Final Value of R0: 6

Step 4. Return:

BX LR returns to the caller with R0 containing the count of bits set to 1.

Chapter 9. Stack

Introduction to the Stack

In previous chapters, we briefly touched upon the stack to preserve registers. In this chapter, we will expand on its utility and hardware implementation.

The stack is a specific region in the main memory (DRAM) used to store temporary data and function call information. Logically, it functions as a Last-In-First-Out (LIFO) container. You can visualize this as a container with a single entry point:

- PUSH: The operation to store an item in the stack.
- POP: The operation to retrieve an item from the stack.

Because of the LIFO nature, the last item pushed into the stack is the first one to be popped out. The item that enters last takes the address pointed to by the stack pointer.

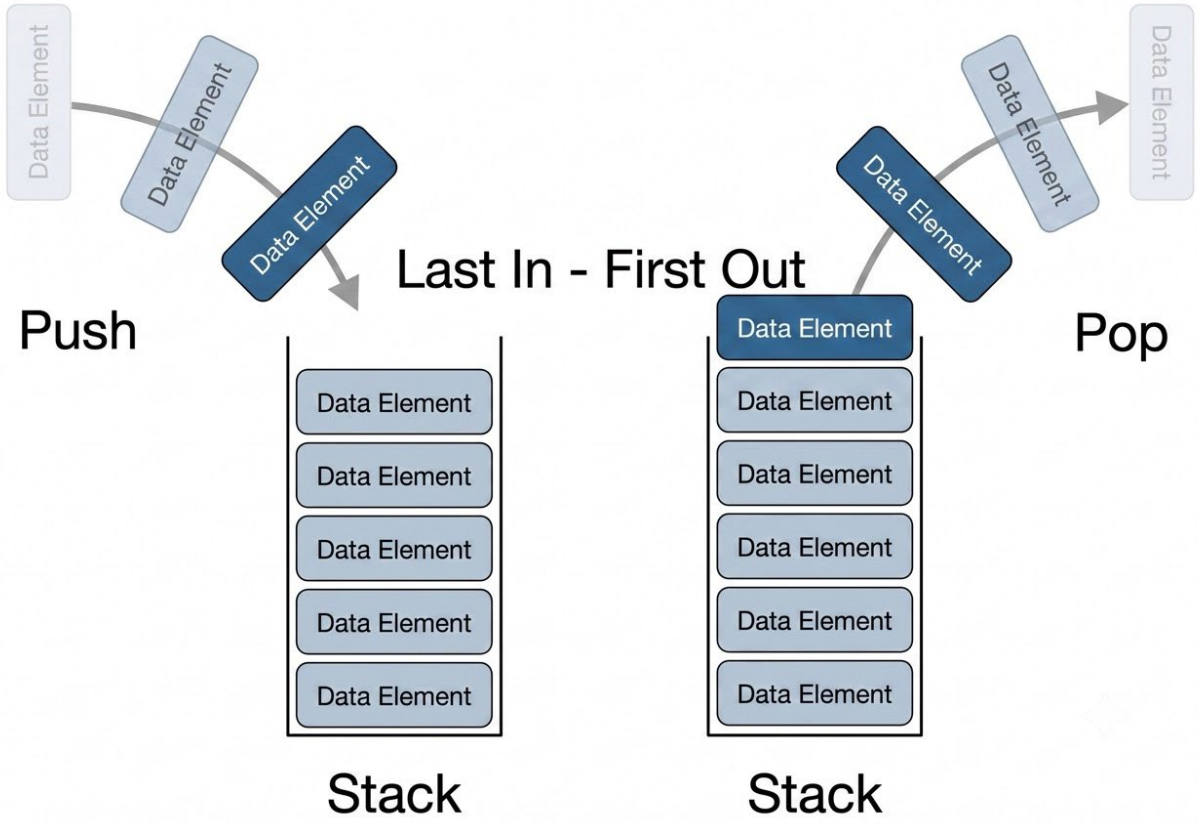


Fig 9.1: Stack illustration

9.1 Architecture and Registers

To navigate functions and memory effectively, the ARM architecture utilizes three specific registers. Understanding how these update line-by-line is crucial for debugging hardware behavior.

9.1.1 The Special Registers

R13 - Stack Pointer (SP): This register stores the address of the stack. It points to the location of the last item pushed.

R14 - Link Register (LR): This register stores the return address. When a function is called, the processor needs to know where to return after the function finishes. The LR keeps a 'link' to the instruction following the function call.

R15 - Program Counter (PC): This register points to the next instruction to be executed. As the processor fetches a line of code from RAM, the PC is already updating to point to the subsequent line.

9.1.2 Stack Growth Direction

A critical concept in ARM architecture is that the stack grows downward in memory, meaning it grows from a high memory address toward a lower memory address. Consequently, the latest item pushed onto the stack resides at the lowest memory address. For example, if the Stack Pointer is at address 0x....16 and we push a 4-byte register, the new item is stored at 0x....12, which is the SP minus 4.

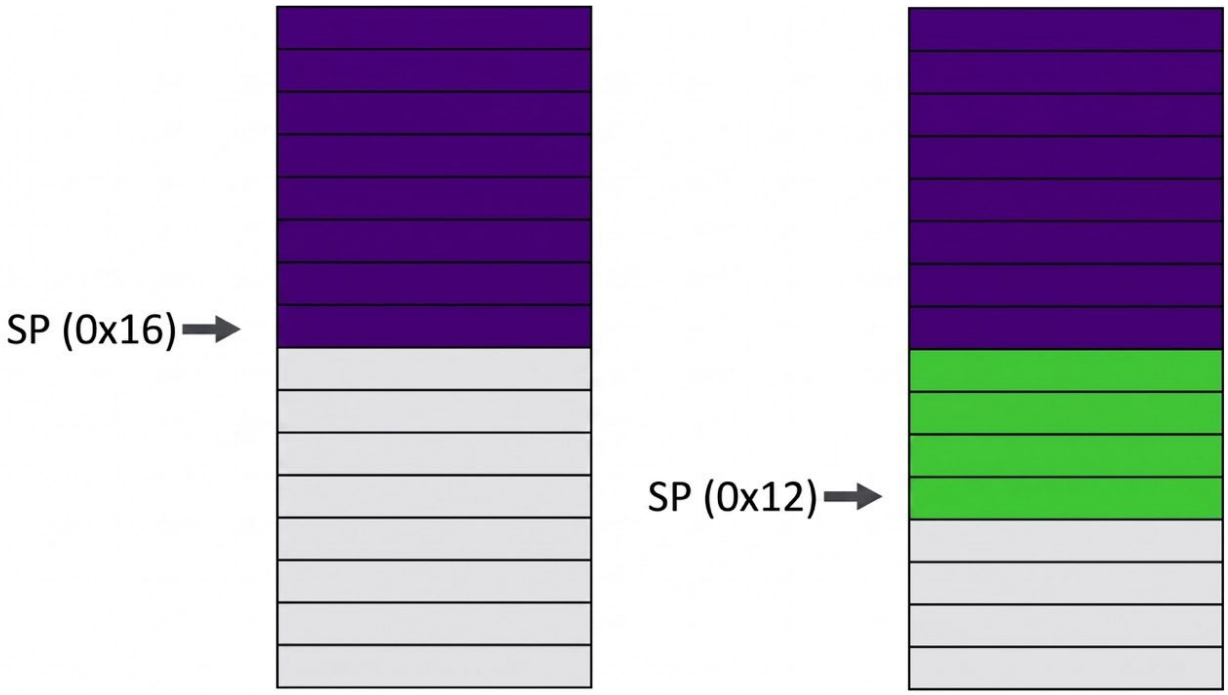


Fig 9.2: Stack Growth

9.2 PUSH and POP Mechanics

When executing a single register operation like PUSH {R5}, the hardware first decrements the Stack Pointer to a lower address and then copies the value of R5 into the memory location now pointed to by the SP. Conversely, when executing POP {R5}, the processor loads the value at the current SP into R5 and then increments the SP to a higher address. ARM also allows pushing multiple registers in a single instruction, where the hardware enforces a strict ordering rule based on the register index, regardless of the order you write them in the assembly code. The register with the highest index is pushed first to the highest relative address in the stack frame, while the register with the lowest index is pushed last to the lowest address, becoming the new top of the stack. For instance, PUSH {R6, R4, R5} is treated exactly the same as PUSH {R4, R5, R6}, meaning R6 is pushed first, followed by R5, and finally R4.

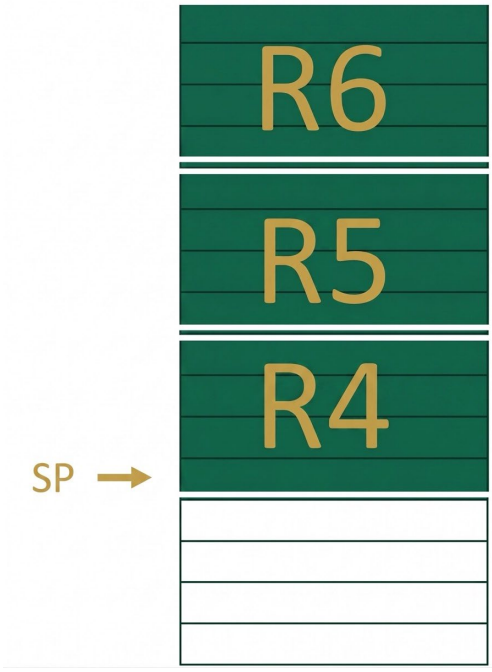


Fig 9.3: Register Index Based PUSH Operation.

9.3 Function Calls: Branch with Link (BL)

In assembly, branching is used to move between sections of code. A standard Branch (B) is an unconditional jump where you go to a label without necessarily planning to return, whereas a Branch with Link (BL) is used for function calls to jump to a subroutine while creating a link to return. When the processor executes a standard instruction, the PC automatically updates to point to the next line, so using a standard branch causes the processor to lose the information about where it came from. The BL instruction solves this by copying the current value of the PC into the Link Register before jumping, allowing the subroutine to know exactly where to return.

9.4 Worked Example: Tracing Memory and Registers

Let us trace the hardware state for a specific code snippet involving a function call and stack operations.

Initial State:

- Stack Pointer (SP) = 0x00000080.
- We assume the code is mapped in RAM starting at 0x20000000.

The Code:

```
0x20000000: BL fn                    // Branch with Link to 'fn'
...
...
0x20001000: MOV R0, #0x12          //(fn label) Move 12 into R0
0x20001004: PUSH {R0, LR, PC}           // Push registers to stack
```

Step-by-Step Execution Trace:

Step 1: Executing BL fn at 0x20000000

When the processor is executing this line, the PC is already looking at the next instruction (0x20000004). The BL instruction copies the PC (0x20000004) into the Link Register (LR). The PC is then updated to the target address: 0x20001000.

Step 2: Executing MOV R0, #0x12 at 0x20001000

R0 is updated with 0x12. The LR remains 0x20000004 (preserving the return link). PC moves to 0x20001004.

Step 3: Executing PUSH {R0, LR, PC} at 0x20001004

We push three registers. Order: PC (R15) -> LR (R14) -> R0.

1. Push PC (0x20001008): SP moves from 0x80 to 0x7C.
2. Push LR (0x20000004): SP moves from 0x7C to 0x78.
3. Push R0 (0x00000012): SP moves from 0x78 to 0x74.

Final Stack Memory Map:

Address	Content	Description
0x80	...	Previous Stack Top
0x7C	0x20001008	Saved PC (High Index)
0x78	0x20000004	Saved LR (Return Address)
0x74	0x00000012	Saved R0 (Low Index / Current SP)

9.5 Variable Scope: Stack vs. Data Segment

9.5.1 Local Variables (Stack)

When a function is called, a stack frame is allocated for it. Variables declared inside the function (e.g., int x = 2;) are stored here.

- Lifetime: They exist only while the function is running.

- Behavior: Once the function returns, the stack pointer moves back up, and those values are effectively 'lost'.

9.5.2 Static Variables (Data Segment)

If a variable is declared as static int x = 2;, it is not stored on the stack.

- Storage: It is stored in the Data Segment.
- Lifetime: It persists for the entire duration of the program.
- Behavior: It 'remembers' the new value between function calls.

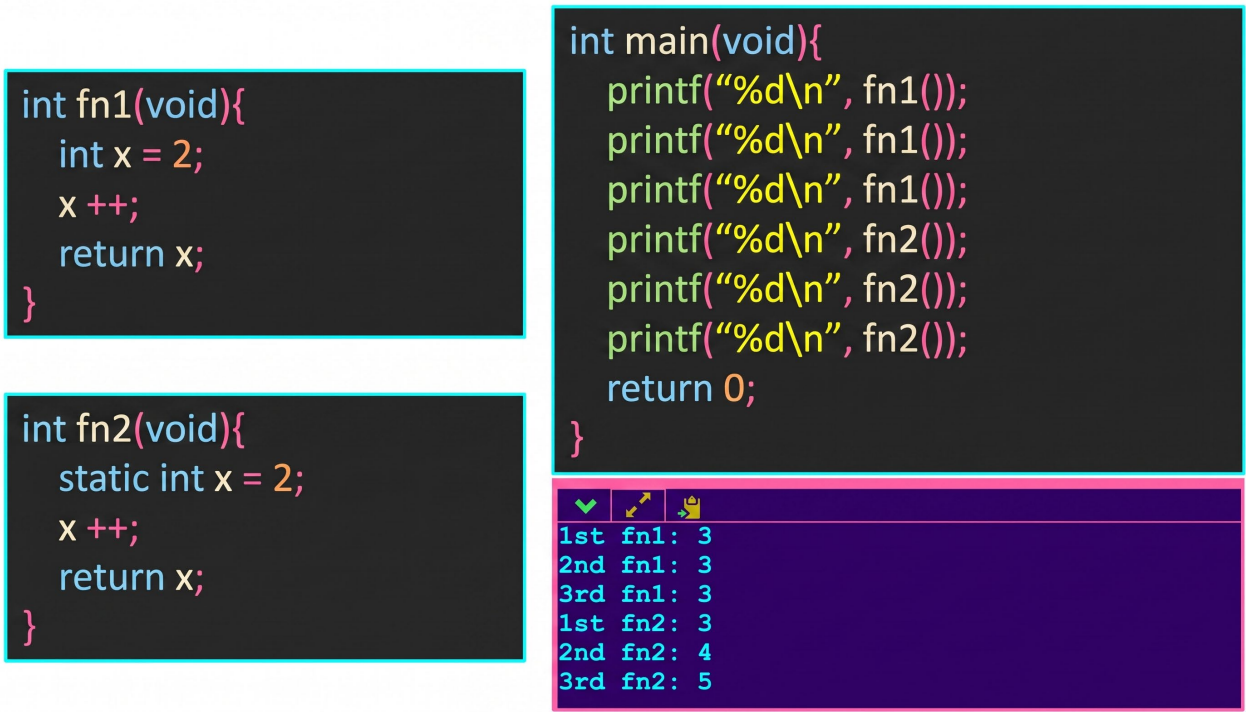


Fig 9.4: Stack vs. Data Segment

9.6 Summary

The stack is a LIFO structure in DRAM that grows downward from high to low memory addresses. The SP (R13) points to the lowest address, representing the top of the stack, while the LR (R14) stores the return address for function calls, and the PC (R15) points to the next instruction to execute. The BL instruction updates the LR with the current PC before jumping to a target. Finally, during a push operation, the highest register number is pushed first, and the lowest register number is pushed last.

Chapter 10. String (Type Char) in Memory and Assembly Implementation

In most programming languages (like C, C++, and others), the char type is used to represent a character (such as a letter, number, or symbol). In memory, a char is typically stored as a byte (8 bits). In this chapter, we will learn how the string is stored in memory and its implementation.

10.1 Type Char in Memory

Type *char* is 8-bit, one memory byte to store a char. It is stored as a 8-bit number (uint8_t), which is called its ASCII code. ASCII is an acronym for American Standard Code for Information Interchange, is a character encoding standard for representing a particular set of 95 (English language focused) printable and 33 control characters – a total of 128 code points. For example,

- Char '0' , ASCII code: 48, then Char '0' = 0b 00110000
- Char '1' , ASCII code: 49, then Char '1' = 0b 00110001
- Char 'A' , ASCII code: 65, then Char 'A' = 0b01000001
- Char 'B' , ASCII code: 66, then Char 'B' = 0b 01000010
- Char 'a' , ASCII code: 97, then Char 'a' = 0b 01100001

10.2 String in Memory

Basically, a string is an array of Char. In C language, string ends with a special Char '\0' (ASCII code is 0). All chars with a string are stored together in memory, each in a memory Byte:

- The first char gets the lowest address
- The last char '\0' gets the highest address

For the address of string, it is the address of its first Char. Because the name of this string is a pointer, pointing to its address.

For example, *char str[]* represents a string named 'str' (an array of char), *str* is a pointer/address, pointing to the first char address of the second char is str + 1, and so on so forth. Implement char str[] with actual string 'Hello world', it could be defined as:

char str[] = "Hello world";

- assume *str* = 0x00002000 (address), therefore * *str* = 'H'
- char (8 bits) occupy one memory slot, and the 'next character' is address + 1
- * (str+4) = str[4] = 'o'

0x0000200B	'\0'
0x0000200A	'd'
0x00002009	'l'
0x00002008	'r'
0x00002007	'o'
0x00002006	'w'
0x00002005	[space]
0x00002004	'o'
0x00002003	'l'
0x00002002	'l'
0x00002001	'e'
0x00002000 (str)	'H'

10.3 Assembly Implementation

In this section, we have several examples of achieving the given C function by assembly implementation.

- strLength
  extern uint32_t strLength(const char str[]);
  This function returns the count of all the characters in str[] except the null character '\0'. For example, strLength("Hello") returns 5.

```
.global strLength          // Make strLength globally accessible
.text                      // Define text (code) section
```

```
strLength:
    MOV R1, R0          // save arg1 address of char in R1
    MOV R0, #0          // set the result register R0 to 0
Start:
    LDRB R2, [R1]           // load char (single byte, 8 bits) from
                            // address (in R1) into R2.
    ADD R1, R1, #1          // update pointer to next char
    CMP R2, #0          // compare char with '\0'
    ADDNE R0, R0, #1    // increase result by 1, if not equal
    BNE Start
    BX LR
```

- strFind

  extern int32_t strFind(const char str[], char c);

  This function returns the position of the first instance of char c in string. It returns -1 if not found. For example, strFind("Hello", 'H') returns 0.

```
.global strFind
.text
strFind:
    MOV R2, R0          // Move the address to R2
    MOV R0, #0          // Move the return index (stored in R0) to 0
Start:
    LDRSB R3, [R2], #1  // Load and increment the address,
                            // Point to the next char. Post-index addressing
                            // for #1
    CMP R3,  #0         // Check if it the tail of string (the null
                        // terminator)
    MOVEQ R0, #-1           // if it's the end and no match found, move -1 to
                        // result
    BEQ End
    CMP R3, R1          // if the char is not '\0', compare current char
                        // with the target char
    BEQ End                 // if match (R3 == R1), return current index by j
                            // jumping to End
    ADD R0, R0, #1          // Increment index after every iteration,
                            // R0 keeps track of the index
```

```
    B Start                 // Continue loop unconditionally. BNE Start also
                            // works, because the only possible condition is
                            // NE ( R3 != R1).
End:
    BX LR
```

- strFindN

extern int32_t strFindN(const char str[], char c, uint32_t n);

This function returns the position of the n-th instance of char c in string, it returns -1 if not found. For example, strFindN("Hello world", 'l', 3) returns 9.

```
.global strFindN
.text
strFindN:
    PUSH {R4}           // Preserve R4
    MOV R3, R0          // Move the address of str to R3
    MOV R0, #0          // Initialize index to 0
Start:
    LDRSB R4, [R3], #1  // Load the next char and increment
                        // address by 'post-indexed addressing'
    CMP R4, #0          // Check if it is the null terminator
    MOVEQ R0, #-1       // if end of string and no match found, return -1
    BEQ End
    CMP R4, R1          // Compare current char with target char
    SUBEQS R2, R2, #1   // if match (R4==R1), decrement count and
                        // updates flags. Executes only if Z is set.
                        // Combines checking and decrementing R2 into
                        // one step
    BEQ End             // if n-th occurrence found, return index
    ADD R0, R0, #1      // Increment index
    B Start             // Continue the loop
End:
    POP {R4}            // Restore R4
    BX LR
```

- strCat

extern void strCat(char strTo[], const char strFrom[]);

This function appends the content of strFrom to strTo

For example, strTo[] = “Hello ”, strFrom[]=“World”, strTo[]=“Hello World”

```
.global strCat
.text
strCat:
    LDRSB R2, [R0], #1          // Load a byte from the source R0 and increment
R0
    CMP R2, #0                  //Compare the loaded byte with null terminator
                                // \0
    BNE strCat                  // If not null, keep scanning for the end of the
                                // string
    SUB R0, R0, #1              // Move R0 back to point at the null terminator
Start:
    LDRSB R3, [R1], #1          // Load a byte from the other string R1 and
                                // increment R1
    STRB R3, [R0], #1           // Store the byte at the current R0 location,
                                // then increment R0
    CMP R3, #0                  // Check if the copied byte is the null terminator
    BNE Start                   // If not, continue copying
    BX LR
```

- Uint32ToBinary

  extern void uint32ToBinary (char str[], uint32_t x)

  This function converts the binary code of x to a string. It stores this string to str[].
  For example, x = 1, str = “00000000000000000000000000000001”

```
.global uint32ToBinary
.text
uint32ToBinary:
    PUSH {R4}
    MOV R2, #0x80000000         // initial mask
Start:
    ANDS R3, R1, R2             // TST
    MOVEQ R4, #’0’              // ‘0’ = #48
    MOVNE R4, #’1’              // ‘1’ = #49
    STRB R4, [R0], #1
    MOVS  R2, R2, LSR #1
    BNE Start
    MOV R4, #0                  // ‘\0’ to R4
```

```
    STRB R4, [R0]
    POP {R4}
    BX LR
```

- BinaryToUint32
  extern uint32_t binaryToUint32 (char str[])
  In contrast, this function converts the null-terminated string (str) back to uint32_t. It treats the string as representing a binary number. Assume the length of str is 32, and str only contains chars '0' and '1' as well as a '\0' at tail.

```
.global binaryToUint32
.text
binaryToUint32:
    MOV R1, R0
    MOV R0, #0
    MOV R2, #0X8000000
Here:
    LDRSB R3, [R1], #1
    CMP R3, #0
    BEQ There
    CMP R3, #'1'
    ORREQ R0, R0, R2
    MOV  R2, R2, LSR #1
    B Here
There:
    BX LR
```

Chapter 11. Array of Integers in Memory and Assembly Implementation

11.1 Memory Layout: Strings vs. Integers

In our previous discussions on strings, we learned that a string is simply a stream of characters stored in consecutive memory addresses. Since a character (char or uint8_t) occupies 1 byte, traversing a string involves adding 1 to the address for each step.

Integer arrays function similarly, but with a critical difference: the 'stride' or step size depends on the data type.

11.1.1 32-bit Integer Arrays (uint32_t)

A standard unsigned integer (uint32_t) occupies 4 bytes, or 32 bits. Consequently, the allocation for an array of N integers requires N x 4 bytes of contiguous memory. When addressing these elements, moving from index i to index i+1 requires increasing the memory address by 4, following the formula

$$\text{Address}(i) = \text{Base} + (i * 4)$$

For example, if an array, my_Arr, starts at the base address 0x2000, the element at index 0 (my_Arr[0]) is located at 0x2000. The subsequent element at index 1 (my_Arr[1]) is found at 0x2004, rather than 0x2001.

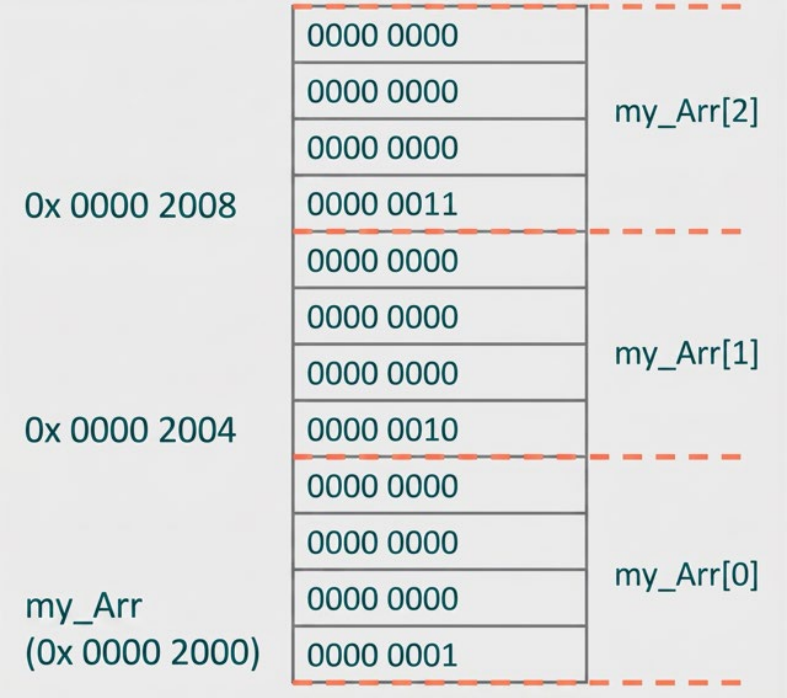


Fig 11.1: Integer array in memory.

11.1.2 16-bit Integer Arrays (uint16_t)

If we declare an array of short integers (uint16_t), each element occupies 2 bytes. Therefore, to move to the next element in the array, we must increase the address by 2, utilizing the formula

$$\text{Address}(i) = \text{Base} + (i * 2)$$

11.2 Iterating with Post-Indexed Addressing

To efficiently loop through arrays in assembly, we employ post-indexed addressing. This specific addressing mode allows us to load a value and update the pointer simultaneously in a single instruction. Using the syntax "*LDR R_dest, [R_ptr], #offset",* the processor first loads the value from the memory address stored in R_ptr into the destination register R_dest. After this load operation completes, it automatically adds the #offset value to R_ptr.

For a 32-bit array, we use an offset of #4. This eliminates the need for a separate ADD instruction to move the pointer.

11.3 Coding Examples

11.3.1 Array Summation (sumU32)

Let's implement a function that sums all elements in a uint32_t array. The function prototype is defined as

extern uint32_t sumU32(const uint32_t x[], uint32_t count);

In our register map, R0 initially holds the array address, and R1 holds the count of elements. The goal of this subroutine is to return the total calculated sum in R0.

Assembly Code:

```
.global sumU32
sumU32:
  MOV R2, R0      // Move array address to R2
  MOV R0, #0      // Initialize sum (accumulator) to 0

Start:
  LDR R3, [R2], #4   // Load value to R3, increment R2 by 4
  ADD R0, R0, R3     // Add value to sum
  SUBS R1, R1, #1    // Decrement count, update flags
  BNE Start          // If count != 0, repeat loop

  BX LR              // Return
```

11.3.2 Scaling an Array (scaleU32)

In this example, we will multiply every element of a source array x by a scalar value and store the result in a destination array y. The function prototype is

extern void scaleU32(uint32_t y[], const uint32_t x[], uint32_t scale, uint32_t count);

Register R0 holds the address of the destination array y, R1 holds the address of the source array x, R2 contains the scaling factor, and R3 holds the element count.

To implement this logic, we need a temporary register to hold the value loaded from x during the multiplication step. We will use R4 for this purpose; however, since R4 is a callee-saved register, we must push it to the stack at the start of the function and pop it at the end to preserve its original value.

Assembly Code:

```
.global scaleU32
scaleU32:
    PUSH {R4}           // Save R4 (callee-saved)

Check:
    CMP R3, #0          // Check if count is 0
    BEQ End             // If 0, exit function

Loop:
    LDR R4, [R1], #4    // Load x[i] into R4, ptr_x++
    MUL R4, R4, R2      // R4 = R4 * scale
    STR R4, [R0], #4    // Store R4 to y[i], ptr_y++
    SUBS R3, R3, #1     // Decrement count
    BNE Loop            // If count > 0, repeat

End:
    POP {R4}            // Restore R4
    BX LR               // Return
```

11.4 Advanced Exercise: Dot Product (dotpU32)

The dot product is a fundamental operation in matrix multiplication that involves multiplying corresponding elements of two arrays and summing the results according to the formula

$$\text{Result} = \text{Sum}(x[i] * y[i])$$

To implement this, you will need two separate pointers to track array x and array y, and you must load two distinct values during every iteration of your loop. Furthermore, you will require multiple temporary registers, such as R4 and R5, to hold the values for the multiplication step. Do not forget to push these callee-saved registers to the stack at the beginning of your subroutine.

Chapter 12. Binary Decoding and Encoding (Floating-point and Decimal)

12.1 Integer (recap)

According to the methods we've learned that converting between binary-encoded (non-negative) integer and decimal

12.1.1 From Binary to Decimal

- List the weights for all the bit positions
- Compare the number with weights starting from MSB.
    - If the number is larger than or equal to the weight, subtract the weight from the number, put '1' as the binary digit in this position, and continue to the next bit on the right with the difference.
    - Otherwise, put '0' and move on to the next bit.

12.1.2 From Decimal to Binary

Method A – Weights Calculation

- Check whether the decimal number is within the range
- List the weights of bits
- Compare the decimal number with weights along the direction from MSB to LSB.
    - If the number is larger than or equal to the weight. Subtract the weight from the number, put '1' at this bit, and continue with the difference.
    - Otherwise, put '0' at this bit and move on to the next bit on the right.

Method B – Division by 2

- Divide the decimal number by 2, record the remainder (0 or 1)
- Take the quotient and continue divide it by 2
- Repeat the division until quotient is 0. The Binary number is the remainders read in reverse order.

12.2 Non-integer (Floating-point)

12.2.1 From Decimal to Binary

To convert a non-integer (Floating-point) decimal format value to its binary representation, we need the following two steps.

1. List the weights at the left and right sides of the point
2. Compare the number (non-integer) with weights from left to right.
    a. If the number is larger than or equal to the weight, subtract the weight from the non-integer, put '1' as the binary digit in this position, and continue to the next bit on the right with the difference.
    b. Otherwise, put '0' and move on to the next bit
    c. Repeat the compare-subtract process until you get 0 left

Practice

Question 1. Convert 68.75 into its binary representation.

Step 1: Convert the integer part (68) to binary.

- 68 ÷ 2 = 34, remainder 0
- 34 ÷ 2 = 17, remainder 0
- 17 ÷ 2 = 8, remainder 1
- 8 ÷ 2 = 4, remainder 0
- 4 ÷ 2 = 2, remainder 0
- 2 ÷ 2 = 1, remainder 0
- 1 ÷ 2 = 0, remainder 1

Binary for 68 is 1000100 (collect remainders from bottom to top).

Step 2: Convert the fractional part (0.75) to binary.

- 0.75 > 0.5 → put 1
- 0.75 – 0.5 = 0.25 >= 0.25 → put 1

Binary for 0.75 is 0.11

Step 3: Combine the integer and fractional parts.

68.75 in binary is 1000100.11

Question 2. Find the binary equivalent of 19.875.

Step 1: Convert the integer part (19) to binary.

- 19 ÷ 2 = 9, remainder 1
- 9 ÷ 2 = 4, remainder 1
- 4 ÷ 2 = 2, remainder 0
- 2 ÷ 2 = 1, remainder 0
- 1 ÷ 2 = 0, remainder 1

Binary for 19 is 10011 (collect remainders from bottom to top).

Step 2: Convert the fractional part (0.875) to binary.

- 0.875 > 0.5 → Put 1
- 0.875-0.5= 0.375 > 0.25 → Put 1
- 0.375-0.25= 0.125>= 0.125→ put 1

Binary for 0.875 is 0.111

Step 3: Combine the integer and fractional parts.

19.875 in binary is 10011.111

Question 3. How to represent 45.625 in binary?

Step 1: Convert the integer part (45) to binary.

45 ÷ 2 = 22, remainder 1

22 ÷ 2 = 11, remainder 0

11 ÷ 2 = 5, remainder 1

5 ÷ 2 = 2, remainder 1

2 ÷ 2 = 1, remainder 0

1 ÷ 2 = 0, remainder 1

Binary for 45 is 101101 (collect remainders from bottom to top).

Step 2: Convert the fractional part (0.625) to binary.

0.625 > 0.5 → Put 1

0.625 – 0.5 = 0.125 < 0.25 → Put 0

0.125 >= 0.125 → Put 1

Binary for 0.625 is 0.101

Step 3: Combine the integer and fractional parts.

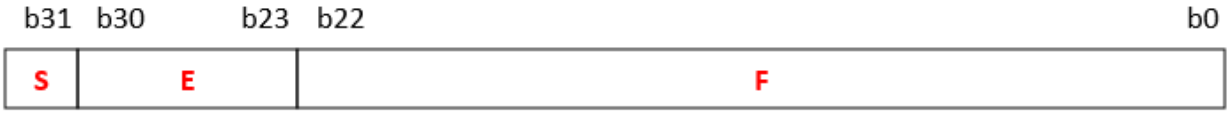
45.625 in binary is 101101.101

12.2.2 Decoding - From Binary to Decimal

To convert a non-integer (Floating-point) binary format value to its decimal representation, we need to follow the IEEE 754 standard, which encodes a number in three parts

- **S - the Sign bit**. 1 bit: Sign (0 for positive or 1 for negative)
- **E - the Exponent.** 8 bits representing a uint8_t:
  - 0 (0b00000000) to 254 (0b11111110) for regular numbers
  - 255 (0b11111111) for non-numeric quantities, e.g., infinity, NaN
- **F - the Fraction (mantissa).** 23 bits representing mantissa.

For example, float type of 32 bits:



Based on the IEEE 754 standard, we calculate the decimal of a floating-point number by the following equation:

$$\text{float number} = (-1)^S \ 2^{E-127} \ 1.F$$

- 1.F represents the binary mantissa with an implicit leading 1
- E - 127 gives the exponent by subtracting the bias. The key reason for the E - 127 part is because the exponent is stored in a biased format. In the biased format, instead of storing the actual exponent (which could be positive or negative), IEEE 754 Standard adds a bias to make the exponent always positive.

  For single-precision floating-point numbers, the exponent is stored as an unsigned 8-bit number, and the bias is 127. This means:
  - If the actual exponent is 0, the stored exponent is 127.
  - If the actual exponent is 1, the stored exponent is 128.
  - If the actual exponent is -1, the stored exponent is 126.

  This bias allows the use of a simple unsigned integer to represent both positive and negative exponents.

Example

Given the 32-bit IEEE 754 floating-point number in Hexadecimal: 0xC2240000. What is the decimal value of this floating-point number?

Step 1. Convert the hex to binary and identify the parts (sign, exponent, mantissa)

0xC2240000 = 1100 0010 0010 0100 0000 0000 0000 0000

In IEEE 754 format, this binary number is divided into three parts:

- **1 sign bit: the first bit ('1' means negative, '0' means positive)**
- **8 bits for the exponent**
- **23 bits for the mantissa (fractional part)**

Step 2. Decode the Exponent

The next 8 bits are the exponent: 10000100.

- **This is an unsigned binary number, so we convert it to decimal:**

10000100 = 132 (base-10)

- **In IEEE 754, the exponent is biased by 127. To get the actual exponent, we subtract 127 from 132.**

- **Actual exponent = 132 - 127 = 5, the actual exponent is 5.**

Step 3: Decode the Mantissa and Convert to Decimal

The mantissa is the next 23 bits (*F*): 01001000000000000000000

In IEEE 754 format, the mantissa has an implicit leading 1. So, we add this to the front then obtain binary representation of the mantissa.

1.01001000000000000000000 (base-2)

Break it to 1 + 0.010010 (base-2), then expand the fractional part as:

- 1st digit after the binary point is 0, so no contribution from 0 x $\frac{1}{2^1}$
- 2nd digit after the binary point is 1, so contribution is 1 x $\frac{1}{2^2} = \frac{1}{4} =$ 0.25
- 3rd digit after the binary point is 0, so no contribution from 0 x $\frac{1}{2^3}$
- 4th digit after the binary point is 0, so no contribution from 0 x $\frac{1}{2^4}$
- 5th digit after the binary point is 1, so contribution is 1 x $\frac{1}{2^5} = \frac{1}{32} =$ 0.03125
- 6th digit after the binary point is 0, so no contribution from 0 x $\frac{1}{2^6}$

(At this point, we can stop as the remaining digits are zeros.)

Then, sum the values of the fractional part: 0.010010 (base-2) = 0.25 + 0.03125 = 0.28125.

Because we need to add the implicit leading 1 before binary point, therefore 1 + 0.28125 = 1.28125 is the final mantissa in decimal.

Step 4: Combine the parts to get the final value

We now have all the components to calculate the final value:

- *S* = 1
- *E* = 132
- *1.F* = 1.28125

Float number = (-1)^1 x 2^(132-127) x 1.28125 = -1 x 32 x 1.28125 = -41

The decimal value of the IEEE 754 floating-point number 0xC2240000 is -41

Practice

Given the 32-bit IEEE 754 floating-point number in hexadecimal

0x40A00000. Decode this value into its decimal representation.

1. Convert Hex to Binary: 0x40A00000 = 0100 0000 1010 0000 0000 0000 0000 0000
2. Break into parts:
    - Sign bit: 0 (positive)
    - Exponent: 10000001 (base-2) = 129
    - Mantissa: 01000000000000000000000 (base-2)
3. Calculate the Exponent: Exponent = 129 – 127 = 2
4. Calculate the Mantissa: Mantissa = 1.01000000000000000000000 (base-2) = 1.25 (decimal)
5. Combine the parts:

1 × 1.25 × 2^2 = 5.0

Thus, the decimal representation of 0x40A00000 is 5.0

12.2.3 Encoding - From Decimal to Binary

To convert a decimal number to its IEEE 754 single-precision floating-point (32-bit):

Step 1. Determine the Sign Bit

- If the number is positive, the sign bit is 0.
- If the number is negative, the sign bit is 1.

Step 2. Convert the Decimal Number to Binary

- Convert the integer part of the number to binary.
- Convert the fractional part to binary by repeatedly multiplying the fraction by 2 and recording the integer part.

Step 3. Normalize the Binary Representation

Convert the binary number to the form 1.F × 2^E.

- Where *F* is the mantissa (fraction) and E is the exponent with bias 127

- Move the binary point such that there is a single 1 to the left of the binary point.
- Count the number of places the binary point was moved to get the exponent.

Step 4. Calculate the Exponent

- In IEEE 754, the exponent is stored with a bias of 127.
- Add 127 to the exponent obtained from the normalization step.
- Convert this value to an 8-bit binary number.

Step 5. Determine the Mantissa

- The mantissa is the fractional part of the normalized binary number *F* (without the leading 1).
- If the number has fewer than 23 bits after the binary point, pad with zeros to get 23 bits.

Step 6. Assemble the IEEE 754 Representation

Combine the sign bit, exponent (8 bits), and mantissa (23 bits) into a 32-bit binary number.

Example

Convert Decimal 13.25 to *IEEE 754* Floating-Point

Step 1: Determine the Sign Bit

- The number is positive, so the sign bit is 0.

Step 2: Convert to Binary

- Integer part: 13(base-10) = 1101 (base-2)
- Fractional part: 0.25 (base-10) = 0.01(base-2)
- Combined: 13.25 (base-10) = 1101.01(base-2)

Step 3: Normalize the Binary Number

- Normalize 1101.01(base-2) to 1.10101(base-2) x 2^3 (in fact, LSL 3). Therefore, exponent is 3.

Step 4: Calculate the Exponent

- Add the bias (127): 3 + 127 = 130 (base-10)

- Convert 130 to binary: 130 (base-10) = 10000010 (base-2)

Step 5: Determine the Mantissa

- The mantissa is the fractional part of 1.10101 (base-2), which is 10101000000000000000000 (padded to 23 bits).

Step 6: Assemble the *IEEE 754* Representation

- Sign bit: 0
- Exponent: 10000010
- Mantissa: 10101000000000000000000

Practice

Question 1. Encode -53.375 to floating-point

- Step 1: un-sign the number (only preserve the absolute value)
  - E.g., -53.375 -> 53.375
- Step 2: convert the absolute value to binary
  - E.g, 53.375 -> 110101.011_2
- Step 3: convert the binary to format 1._____ $\cdot 2^n$
  - Hint: times 2^n means shift to the left by n bits if n is positive; shift to the right by |n| bits if n is negative
  - E.g, $110101.011_2 = 1.10101011_2$ LSL 5 = $1.10101011_2 \cdot 2^5$
- Step 4: get S, E, and F from equation
  - Sign: 1 for negative
  - Exponent excess: $2^5 = 2^{E-127}$ -> E (8-bit) =132= 0b1000 0100
  - Fractional Mantissa: 1.10101011_2 -> F (23-bit) = 1010 1011 0000 0000 0000 000
- Step 5: put them together
  - 0b 1100 0010 0101 0101 1000 0000 0000 0000 = 0xC2558000

Question 2. Encode 0.375 to floating-point

- Step 1: un-sign the number (only preserve the absolute value)
  - E.g., 0.375 -> 0.375
- Step 2: convert the absolute value to binary
  - E.g, 0.375 -> 0.011_2
- Step 3: convert the binary to format 1._____ $\cdot 2^n$

    - Hint: times 2^n means shift to the left by n bits if n is positive; shift to the right by |n| bits if n is negative
    - E.g, $0.011_2 = 1.1_2$ LSR $2 = 1.1_2 \cdot 2^{-2}$
- Step 4: get S, E, and F from equation
    - Sign: 0 for positive
    - Exponent excess: $2^{-2} = 2^{E-127}$ -> -2 = E -127 = E =125 (decimal) = 0b 0111 1101 (8-bit)
    - Fractional Mantissa: 1.1_2 -> F (23-bit) = 1000 0000 0000 0000 0000 000
- Step 5: put them together
    - 0011 1110 1100 0000 0000 0000 0000 0000 = 0x3EC00000

Chapter 13. Data Alignment (Packed and Unpacked) and Assembly Implementation

In ARM, key components in ARM memory architecture include: Registers, Stack and Heap, Code Segment, and Data Segment. Memory system is managed in the unit of ***Word*** (4 Byte = 32 bit). The address of ***Word*** is a multiple of 4 (Each Word contains four Bytes), CPU can access 32 bit or 4 Byte in one Word simultaneously (within one reading cycle). For example, based on memory alignment that CPU need two cycles to read Byte 3-4 or Byte 11-14, although they are consecutive. Memory is typically accessed in aligned blocks, such as 4-byte or 8-byte chunks, depending on the CPU architecture.

For the Example1 shown below, reading bytes 3-4 could cross from one aligned 4-byte boundary (bytes 0–3) to another (bytes 4–7).

Word 4	16	17	18	19
Word 3	12	13	14	15
Word 2	8	9	10	11
Word 1	4	5	6	7
Word 0	0	1	2	3

For the other Example2, it takes two reading cycles to read addr+37 ~ addr+40. By the solution of unpacked alignment and padding, we add three blank (zero) Bytes to replace addr+37 ~ addr + 39, then the new alignment fits one reading cycle efficiently.

addr + 40	addr + 41	addr + 42	addr + 43
addr + 36	addr + 37	addr + 38	addr + 39

addr + 40	addr + 41	addr + 42	addr + 43
addr + 36	Zero Padding	Zero Padding	Zero Padding

Therefore, there are two data alignment strategies in structure: Unpacked and Packed.

In Unpacked Alignment, offsets are adjusted to align each field, leading to a larger total size (e.g., with padding between fields). Access to fields is faster, but the structure size increases due to padding.

In Packed Alignment, each field directly follows the previous one, resulting in a smaller size with minimal or no padding. Access to fields may be slower due to misalignment, which can impact performance on alignment-sensitive architectures.

In a structure, the offset of a member is the number of bytes from the beginning of the structure to the start of that particular member. Offset of a member means the address difference between the member and the structure (the first member).

The offsets represent where each member is stored in memory relative to the beginning of the structure (e.g., taxId being at offset 0). Each offset is the cumulative sum of the previous fields' sizes, as there is no padding in this packed structure.

13.1 Unpacked Structure in Memory

Assume a structure in memory that called BUSINESS, which includes the following fields. Based on the Unpacked Alignment, let's take a look at how we determine the offsets and padding size step by step.

```
typedef struct _BUSINESS
{
        uint32_t taxId; // 4 Bytes
        char name[33]; //33 Bytes
        uint32_t addNo; // 4 Bytes
        char direction; // 1 Bytes
        char street[33]; // 33 Bytes
        char city[33]; // 33 Bytes
        char state[3]; // 3 Bytes
        uint32_t zip; //4 Bytes
} BUSINESS;
```

1. **taxId (4 bytes)**
- **Offset: 0 (aligned at the start of the structure)**
- No padding is needed here.
- Offset after taxId: **4**

2. **name[33] (33 bytes)**
- **Offset: 4**
- Padding (3 bytes) is added here to align the next *uint32_t addNo* to a 4-byte boundary. Ends at Offset **4 + 33 = 37 + 3 (padding) = 40**.

3. **addNo (4 bytes)**
- **Offset: 40**
- No padding is needed here since addNo is already aligned to 4 bytes.
- Offset after addNo: **40 + 4 = 44**

4. **direction (1 byte)**
- **Offset: 44**

- direction is 1 byte, so no padding will be added due to ***char*** type has 1-byte alignment. Offset after *direction*: **44 + 1 = 45**

5. **street[33] (33 bytes)**

- **Offset: 45**
- street is 33 byte, so no padding will be added due to ***char*** type has 1-byte alignment.
- Offset after *street*: **45 + 33 = 78**

6. **city[33] (33 bytes)**

- **Offset: 78**
- city is 33 byte, so no padding will be added due to ***char*** type has 1-byte alignment.
- Offset after *city*: **78 + 33 = 111**

7. **state[3] (3 bytes)**

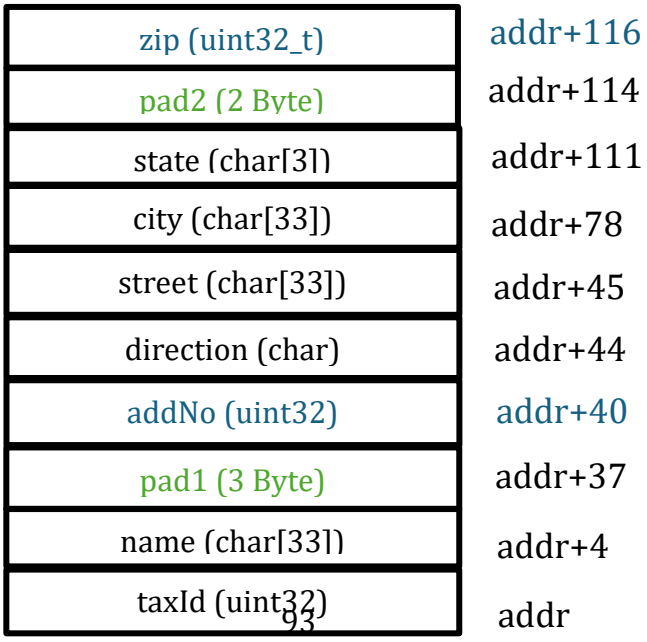
- **Offset: 111**
- **Padding (2 byte) is added here to align the next uint32_t to a 4-byte boundary.**
- Offset after state: **111 + 3 = 114 + 2 (padding) = 116**

8. **zip (4 bytes)**

- **Offset: 116**
- No padding is needed here since zip is already aligned to 4 bytes.
- Offset after zip: **116 + 4 = 120**

Therefore, the total size of BUSINESS in Unpacked Alignment:

4 Bytes (taxId) + 33 (name) + 3 (first padding - pad1) + 4 (addNo) + 1 (direction) + 33 (street) + 33 (city) + state (3) + 2 (second padding - pad2) + 4 (zip) = 120 Bytes



13.2 Packed Structure in Memory

Assume a structure in memory that called BUSINESS, which includes the following fields.

The size of a packed structure in C or C++ is the sum of the sizes of its individual fields with minimal or no padding. When a structure is packed, it minimizes padding bytes that would normally be added for memory alignment, resulting in a smaller total size.

```
typedef struct _BUSINESS
{
        uint32_t taxId; // 4 Bytes
        char name[33]; //33 Bytes
        uint32_t addNo; // 4 Bytes
        char direction; // 1 Bytes
        char street[33]; // 33 Bytes
        char city[33]; // 33 Bytes
        char state[3]; // 3 Bytes
        uint32_t zip; //4 Bytes
} BUSINESS;
```

Since the structure is packed, offsets will directly follow the size of each field, with no padding. Here's the breakdown of offsets for each member:

1. **taxId (4 bytes)** Offset = 0 (the start of the structure)
2. **name (33 bytes)** Offset = 4 (immediately after taxId which is 4 bytes)
3. **addNo (4 bytes)** Offset = 4 + 33 = 37 (immediately after name which is 33 bytes)
4. **direction (1 byte)** Offset = 37 + 4 = 41 (immediately after addNo which is 4 bytes)
5. **street (33 bytes)** Offset = 41 + 1 = 42 (immediately after direction which is 1 byte)
6. **city (33 bytes)** Offset = 42 + 33 = 75 (immediately after street which is 33 bytes)
7. **state (3 bytes)** Offset = 75 + 33 = 108 (immediately after city which is 33 bytes)
8. **zip (4 bytes)** Offset = 108 + 3 = 111 (immediately after state which is 3 bytes)

Therefore, total size of BUSINESS (Packed Alignment):

4 Byte (taxId) + 33 (name) + 4 (addNo) + 1 (direction) + 33 (street) + 33 (city) + state (3) + 4 (zip) = 115 Byte

13.3 Comparison Between Unpacked and Packed Cases

Using the same example of BUSINESS structure, comparing the Unpacked Alignment and Packed Alignment:

Aspect	Unpacked	Packed
Total Size	120 bytes	115 bytes
Alignment	Fields aligned to 4 bytes	Fields follow each other exactly
Padding	Yes	No padding
Performance	Faster due to alignment	Potentially slower due to misalignment on certain architectures

Table 13. 1 Difference between Unpacked and Packed structures

Unpacked structures include padding to align fields to 4-byte boundaries, resulting in a larger size (120 bytes in this case) but potentially faster access.

Packed structures eliminate padding, minimizing size (115 bytes) but potentially reducing access speed due to misalignment on architectures that require aligned accesses.

Chapter 14. VFP Instructions (VLDR, VSTR, VMOV, and VCVT) and Assembly Implementation

14.1 The VFP Coprocessor

In the previous chapters, we focused on integer arithmetic using the standard ARM Core registers (R0–R15). However, many applications require greater precision or the ability to represent fractional numbers. To handle this, the ARM architecture utilizes a hardware component known as the Vector Floating Point (VFP) Coprocessor.

The VFP is a coprocessor, meaning it works alongside the main CPU but has its own distinct resources. It provides hardware support for single-precision (32-bit) and double-precision (64-bit) floating-point arithmetic.

14.2 VFP Register Organization

Because the VFP is a separate unit, it does not use the standard R registers for its operations. Instead, it has a dedicated register bank.

14.2.1 Register Banks

- **Single-Precision Registers (S):** There are 32 registers labeled S0 through S31. Each is 32 bits wide and used to store single-precision floats.
- **Double-Precision Registers (D):** There are 16 registers labeled D0 through D15. Each is 64 bits wide and used to store double-precision values.

14.2.2 Register Aliasing

It is critical to understand that the S and D registers are aliased over the same physical memory. They are not separate storage locations. A double-precision register is formed by combining two consecutive single-precision registers.

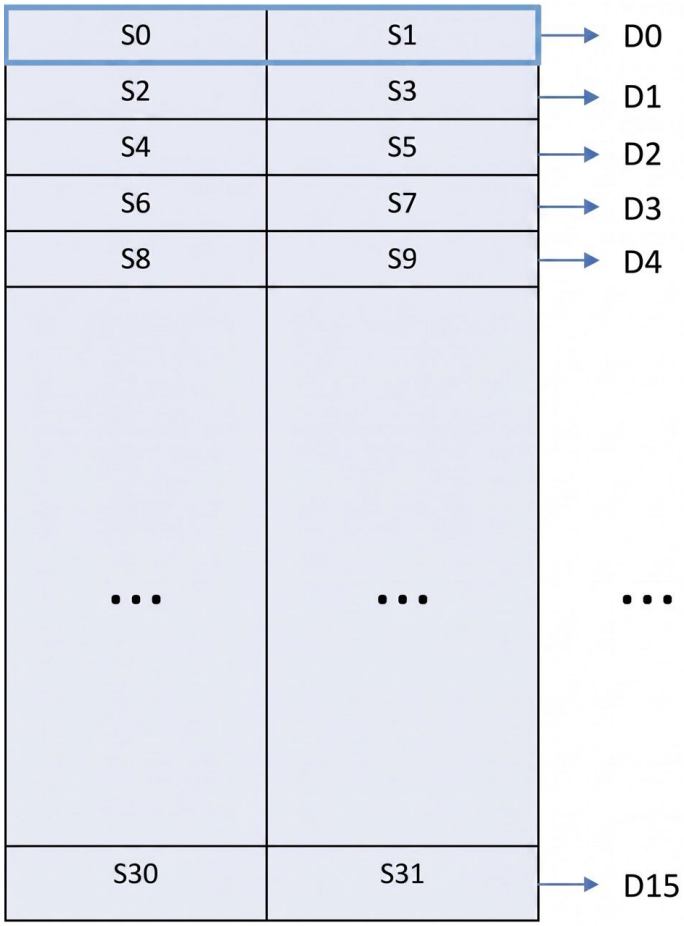


Fig 14.1: VFP Register Aliasing.

- D0 occupies the same bits as S0 and S1.
- D1 occupies the same bits as S2 and S3.
- D15 occupies the same bits as S30 and S31.

Implications for Programming: If you store a value in S0, you have partially overwritten D0. Consequently, you cannot use D0 to store a double at the same time. Conversely, if you store a double in D0, both S0 and S1 are occupied.

14.3 Procedure Call Standard (PCS) for VFP

When calling functions that mix integer and floating-point arguments, we must follow strict conventions to ensure arguments are passed in the correct registers.

14.3.1 Argument Passing Rules

- **Integers:** Arguments no larger than 32 bits (e.g., int, char, uint32_t, pointers) are loaded into the first available Core register (R0–R3).

- **Single-Precision Floats:** Loaded into the first available S register.
- **Double-Precision Floats:** Loaded into the first available D register.

Return Values:

- Integer results: Returned in R0.
- Float results: Returned in S0.
- Double results: Returned in D0.

14.3.2 Mixed Argument Example

Consider the following C function signature:

uint32_t fn(float a, double b, float c, float d)*

The register allocation follows the order of arguments:

1. **float a:** Goes to S0 (First available S register).
2. **double b:** Goes to D1.
   Why D1? D0 is formed by S0 and S1. Since S0 is occupied by argument 'a', D0 is considered unavailable. The next free double register is D1.
3. **float* c:** Goes to R0.
   Why R0? Even though it points to a float, the pointer itself is a memory address (an integer).
4. **float d:** Goes to S1.
   Why S1? S0 is used by 'a'. D1 uses S2 and S3. This leaves S1 (the upper half of D0) empty and available for a single-precision argument.
5. **Return Value:** The function returns a uint32_t, so the result will be in R0.

14.4 VFP Instructions

VFP instructions generally use a V prefix. You must often specify the data size using a suffix: .F32 for single-precision or .F64 for double-precision.

14.4.1 Load and Store (VLDR, VSTR)

These instructions move data between RAM and VFP registers.

- VLDR.F32 S0, [R0] – Load 32 bits from address R0 into S0.
- VLDR.F64 D0, [R0] – Load 64 bits from address R0 into D0.

Constraint: Unlike standard integer instructions, VLDR and VSTR do not support post-indexed addressing (e.g., VLDR.F32 S0, [R0], #4 is illegal). You must handle the address increment manually using a separate ADD instruction.

14.4.2 Moving Data (VMOV)

The VMOV instruction transfers raw bits between registers. It does not perform data conversion.

- VMOV.F32 S0, R0: Copies the 32 bits from R0 directly to S0.
- VMOV.F64 D0, R2, R3: Copies 64 bits (from R2 and R3) into D0.

Important: If R0 holds the integer 1, VMOV S0, R0 will result in S0 holding the floating-point value 0.000...01 (a denormalized number), not 1.0. The bits are simply copied, not re-encoded.

Immediate Values: You cannot move an immediate value directly into a VFP register. You must first move the value into an R register and then VMOV it to the VFP register.

14.4.3 Data Conversion (VCVT)

To change the data type (e.g., Integer to Double) while preserving the numeric value, use VCVT.

- Syntax: VCVT.<target_type>.<source_type>
- Example: VCVT.F64.U32 D5, S7
  This treats the bits in S7 as an unsigned 32-bit integer, calculates the equivalent double-precision value, and stores it in D5.

14.4.4 Arithmetic Operations

- VADD.F64 D0, D0, D1 (Add)
- VSUB.F32 S0, S1, S2 (Subtract)
- VMUL.F64 D2, D2, D0 (Multiply)
- VDIV.F32 S0, S1, S2 (Divide)

14.5 Coding Examples

14.5.1 Array Summation (sumF64)

Task: Implement double sumF64(const double x[], uint32_t count). Input: Array address in R0, count in R1. Output: Sum returned in D0.

```
.global sumF64
sumF64:
    MOV R2, #0              // Prepare integer 0
    VMOV D0, R2, R2         // Initialize sum (D0) to 0.0 by
                           // copying zero bits
                           // (Integer 0 is same bit pattern as
                           // Double 0.0)

LoopCheck:
    CMP R1, #0              // Check if count is 0
    BEQ End                 // If count is 0, exit

    VLDR.F64 D1, [R0]       // Load double from array x to D1
    ADD R0, R0, #8          // Manually increment pointer by 8
                           // bytes (size of double)
                            // Post-indexing is not supported!
    VADD.F64 D0, D0, D1     // Sum = Sum + current value
    SUBS R1, R1, #1         // Decrement count
    B LoopCheck             // Repeat

End:
    BX LR                   // Return
```

14.5.2 Calculating Mean (meanF64)

Task: Implement double meanF64(const double x[], uint32_t count). The count (R1) is an integer, but the sum (D0) is a double. We must convert the count to a double before dividing.

```
.global meanF64
meanF64:
    MOV R2, #0
    VMOV D0, R2, R2         // Initialize sum D0 = 0.0

                             // CONVERSION STEP
    VMOV S4, R1             // 1. Move integer count bits from
                           // R1 to S4
```

```
    VCVT.F64.U32 D2, S4     // 2. Convert unsigned int in S4 to
                            // double in D2 Now D2 contains the count
                            // as a double value

    CMP R1, #0              // Safety check: Avoid division by zero
    BEQ End                 // If count is 0, return 0.0

Loop:
    VLDR.F64 D1, [R0]       // Load current double
    ADD R0, R0, #8          // Advance pointer by 8 bytes
    VADD.F64 D0, D0, D1     // Add to sum
    SUBS R1, R1, #1         // Decrement loop counter
    BNE Loop                // Repeat if count not zero

                            // DIVISION STEP
    VDIV.F64 D0, D0, D2     // Result = Sum (D0) / Count (D2)

End:
    BX LR                   // Return
```

Chapter 15. Clock Cycle and Pipeline in ARM

15.1 Clock Cycle

Clock cycle is the basic time unit of CPU. Any action consumes one or multiple clock cycle, clock frequency (represented by F) determines clock cycle (represented by T).

$$T = \frac{1}{F}$$

$$F\text{: clock frequency}$$

For example, assume a CPU model of i7-8700 with 3.20GHz, what's the clock cycle?

First of all, we know that frequency unit 1GHz equals 10^9 Hz, time unit 1 nanosecond (ns) equals 10^{-9} seconds. Therefore, insert the CPU frequency and convert the units we can get:

$$T = \frac{1}{3.2 \times 10^9}\ \text{s}$$

$$= 0.3125\ \text{ns}$$

This CPU has clock frequency of 0.3125 ns.

15.2 Three-Stage Execution of Instructions

There are three stages execution of instructions, each stage takes 1 clock cycle of CPU

Stage 1 - Fetch (F): The fetcher load the instruction (32-bit machine code; object file) from Mem to CPU (instruction register)

- Simply copy the next line of instruction without looking at its content
- This stage knows nothing about the instruction

Stage 2 - Decode (D): The decoder decode the instruction

- Set control signals according to the instruction in the instruction register
- Check whether this instruction disrupt the sequential workflow (whether this instruction is a branch)
    - If yes, inform fetcher of the next instruction to fetch.
- E.g., ADDS R1, R2, R3 -> CTRL1 = 0b0010; CTRL2 = 0b0011; WR1 = 1; clear rest WR

Stage 3 - Execute (X): The Executor executes the instruction

Data transfer, logical and arithmetic operations all take place during this stage.

Therefore, the first conclusion is three clock cycles are needed to execute one instruction, because **C1 (F1) – C2 (D1) – C3 (X1)**. To be noted, C1 represents the first clock cycle, F1 represents Fetch the first instruction, and so on.

Following the logic, what about two instructions?

Typically, C1 (F1) -> C2(D1) -> C3(X1) -> C4 (F2) -> C5 (D2) -> C6 (X2), in total six cycles based on the three-stage execution of instructions. But do you really need six clock cycles?

In fact, CPU has separate components for F, D, and X, and they work independently. In C2, fetcher is free to fetch the second instruction:

C1 (F1) -> C2(D1; F2) -> C3(X1, D2) -> C4 (X2)

Therefore, the second conclusion is four clock cycles are needed to execute two instructions. For three instructions, only five clock cycles are needed.

C1 (F1) -> C2(D1; F2) -> C3(X1, D2, F3) -> C4 (X2, D3) -> C5(X3)

15.3 Pipeline in ARM

Given an example of assembly code:

```
Line 1.     MOV R0, #1
            Start:
Line 2.         ADD  R1, R0, R0
Line 3.         SUB   R0, R1, R0
Line 4.         B Start
Line 5.         MOV R0, #4
Line 6.         BX LR
```

The pipeline of execution in ARM could be described as the following table:

	Clock 1	**Clock 2**	**Clock 3**	**Clock 4**	**Clock 5**	**Clock 6**	**Clock 7**	**Clock 8**
Fetcher	F-1	F-2	F-3	F-4	F-5	F-2	F-3	F-4
Decoder		D-1	D-2	D-3	D-4	Null	D-2	D-3
Executer			X-1	X-2	X-3	X-4	Null	X-2

For most assembly instructions (ADD, SUB, MOV, etc.), 1 clock cycle is needed to execute, which means this instruction occupies the **Executor** (X) for 1 clock cycle. According to the table,

In Clock 5:

- Decode 'B start' instruction
- The next instruction is line 2
- Inform fetcher about the next instruction through control signal
- Expect line 2 from fetcher
- but fetcher fetches line 5 to the instruction register because on the Fetch stage, the exact instruction still is unknown

In Clock 6:

- **Fetcher** (F) fetch the line 2
- Decoder in clock 6:
- Line 5 in instruction register because it was fetched in clock 5
- Nothing (**Null**) for decoder to decode since it expect line 2, which is not ready until the next cycle

In Clock 7:

- **Decoder** (D) Get line 2 from fetcher and decode it
- **Executer** (X) has nothing (**Null**) to execute because decoder decoded nothing in clock 6

In fact, Line 5 and Line 6 will never be reached or executed in this program due to the infinite loop caused by the B Start instruction.

Beyond most of assembly instructions we mentioned above, we can summarize that for Branch always (B, BL, BX) instructions, in total 2 clocks (1 clock to execute + 1 imputed clock penalty) are needed for execution.

Given the other example of assembly code

```
Line 1.     MOV R0, #1
Line 2.     MOV R1, #3
            Start:
Line 3.         ADD  R0, R0, R0
Line 4.         SUBS  R1, R1, #1
```

Line 5. BNE Start

Line 6. MOV R0, R1

Line 7. MOV R1, #0

Line 8. BX LR

	Clock 1	**Clock 2**	**Clock 3**	**Clock 4**	**Clock 5**	**Clock 6**	**Clock 7**	**Clock 8**	**Clock 9**	**Clock 10**
Fetcher	F-1	F-2	F-3	F-4	F-5	F-6	F-7	F-3	F-4	F-5
Decoder		D-1	D-2	D-3	D-4	D-5	Null	Null	D-3	D-4
Executer			X-1	X-2	X-3	X-4	X-5	Null	Null	X-3

In Clock 6:

Decode line 5 'BNE Start'

- It may be a branch, depending on Z flag
- But Z flag is determined by X-4 and unknow until the end of this cycle
- Decoder cannot inform fetcher which instruction to fetch
- Fetcher will fetch the next line (line 7) in the next cycle

In Clock 7:

- Check Z flag: clear, so it's a branch true, the next instruction is line 3
- Inform fetcher to fetch line 3 in the next clock
- Right now, line 6 in instruction register
- Fetcher is copying line 7 to the instruction register
- Decoder has nothing to decode because the expected line 3 is not fetched yet

In Clock 8:

- Right now, line 7 in instruction register
- **Decoder** has nothing (**Null**) to decode
- **Executor** has nothing (**Null**) to execute because decoder decoded nothing in clock 7

In Clock 9:

- **Decoder** decodes line 3
- **Executor** has nothing (**Null**) to execute because decoder decoded nothing in clock 8

In addition of Branch Always instructions, we can summarize that for Branch Conditionally (Bcond) instructions, if the condition check is **True**, in total **3 clock cycles** (1 clock to execute + 2 imputed clock penalty) are needed for execution.

To continue the execution,

In Clock 16:

- Decode line 5 'BNE Start'
- It may be a branch, depending on Z flag
- But Z flag is determined by X-4 and unknow until the end of this cycle
- Decoder cannot inform fetcher which instruction to fetch
- Fetcher will fetch the next line (line 7) in the next cycle

In Clock 17:

- Check Z flag: set, so it's a branch false, the next instruction to decode is line 6
- Right now, line 6 is in instruction register. Perfect!
- Check Z flag: set, so it's a branch false, the next instruction is line 6
- Decode line 6 since it was fetched in Clock 16

At the end of this section, we can summarize that for Branch conditionally (Bcond) instructions, if the condition check is **False**, only **1 clock cycle** is needed for execution.

15.3 Analysis of Clock Cycles

Given 5 GHz clock, calculate the time of execution from *BL fn* to *BX LR*

```
        BL fn
                    ...
... multiple irrelevant instructions ...
        fn:         ...
          MOV R0, #1
          MOV R1, #3
        Start:
          ADD  R0, R0, R0
          SUBS R1, R1, #1
          BNE start
          BX LR
```

According to the pipeline of ARM and clocks cycles for different instructions, we solve this question by the following three steps.

Step 1. Count the number of executions per instruction

BL fn	Executes once (transfers control to the function fn), and it takes 2 clock cycles.
fn:	
MOV R0, #1	Executes once, takes 1 clock cycle.
MOV R1, #3	Executes once, takes 1 clock cycle.
Start:	
ADD R0, R0, R0	Executes 3 times,R1 starts at 3 and the loop decrements R1 each time
	Executes 3 times,R1 starts at 3 and the loop decrements R1 each time
SUBS R1, R1, #1	Executes 3 times. The first two times, the branch is taken (since R1 is not zero). The third time, the branch is not taken after SUBS updated Z flag, then take exit from the loop. Each branch that is taken takes 3 clock cycles, while the branch not taken, BNE start takes 1 clock cycle.
BNE start	
	Executes **once**, taking **2 clock cycles** to return from the function.

Step 2. Calculate total clock cycles

BL fn	BL fn: 1 (once) × 2 = 2 clock cycles
fn:	
MOV R0, #1	MOV R0, #1: 1 × 1 = 1 clock cycle
MOV R1, #3	MOV R1, #3: 1 × 1 = 1 clock cycle
Start:	
ADD R0, R0, R0	ADD R0, R0, R0: 3 (three times) × 1 = 3 clock cycles
SUBS R1, R1, #1	SUBS R1, R1, #1: 3 × 1 = 3 clock cycles
BNE start	BNE Start: 2 (twice, first two iterations) × 3 + 1 (once for last iteration) × 1 = 7 clock cycles (2 branches taken and 1 branch not taken)
BX LR	BX LR: 1 × 2 = 2 clock cycles

Total Clock Cycles: 2 + 1 + 1 + 3 + 3 + 7 + 2 = 19 clock cycles

Step 3. Calculate execution time

Given a 5 GHz clock, the clock period is:

Clock Period = 1 / 5 GHz = 1 / (5 x 10^9) seconds = 0.2 ns

Total execution time: Execution Time = 19 × 0.2 ns = 3.8 ns

The total time of execution from BL fn to BX LR is 3.8 nanoseconds.

Chapter 16. Physical Memory (RAM), Cache, and Memory Virtualization

16.1 The Memory Bottleneck

In the previous chapters, we maximized CPU execution speed using pipelining. However, we now face a fundamental physical limitation known as the Von Neumann Bottleneck. The architecture of modern computers separates the Processing Unit (CPU) from the Storage Unit (Memory), creating a massive performance disparity.

- **CPU Speed:** A modern processor cycle is extremely fast, often less than 1 nanosecond (e.g., ~0.3 ns for a 3.2 GHz CPU).
- **RAM Speed:** Accessing Main Memory is significantly slower, taking around 60 nanoseconds.

This creates a "Time Gap" where the CPU is roughly 200 times faster than the memory. If the CPU had to fetch every instruction and data byte directly from RAM, it would spend the vast majority of its time idle. Standard architecture bridges this gap using Cache. Cache is an extra storage hardware of fast speed but small size. The cache stores a copy of the data that is likely to be accessed soon.

16.2 The Principle of Locality

Ideally, we would build the entire main memory out of high-speed SRAM (the technology used in caches), but it is too expensive and power consuming. Instead, we use a small amount of fast memory to store only the "important" data.

But how does the hardware predict what data is "important"? It relies on the Principle of Locality, which observes that programs function in predictable patterns.

16.2.1 Temporal Locality

Definition: Programs tend to access the same data or instruction repeatedly over a short period.

Example: Consider a for loop running 1000 times. The instructions defining that loop are fetched and executed 1000 times in rapid succession. Similarly, the loop counter variable 'i' is updated constantly. Because this data is used repeatedly in a short time window, it should be kept in the fastest memory available close to the CPU.

16.2.2 Spatial Locality

Definition: Programs tend to access data located near each other in the physical memory space.

Example: If a program accesses Array[0], it is statistically highly likely that it will access Array[1] next. Similarly, instruction execution usually proceeds sequentially. Therefore, when the hardware fetches one specific address, it also fetches the neighboring data, anticipating that the neighbors will be needed soon.

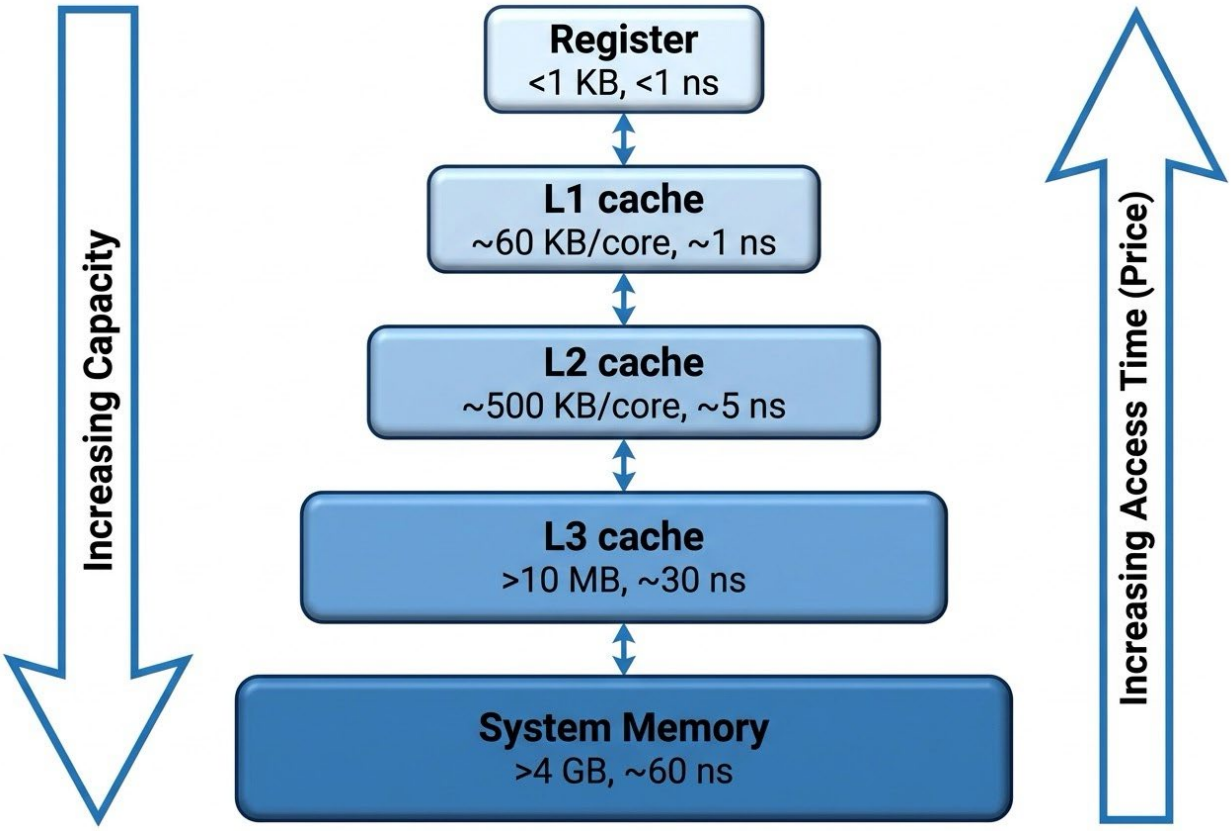


Fig 16.1: Cache Hierarchy

16.3 Cache Memory Hierarchy

To balance the trade-off between Cost, Capacity, and Speed, modern systems use a Multi-Level Cache Architecture.

16.3.1 The Hierarchy Levels

1. Registers: The fastest storage (<1 ns), located inside the CPU. Tiny capacity (<1 KB).
2. L1 Cache: Very fast (~1 ns), small size (~64 KB). The first place the CPU looks.
3. L2 Cache: Moderately fast (~5 ns), medium size (~500 KB).
4. L3 Cache: Slower (~20 ns), large shared size (>10 MB).
5. System Memory (RAM): Slowest (~60 ns), huge capacity (>4 GB).

16.3.2 The Inclusive Subset Property

Crucially, the memory hierarchy operates on an inclusive subset principle. This creates a chain of redundancy to ensure speed:

- L3 holds a subset of data from RAM.
- L2 holds a subset of data from L3.
- L1 holds a subset of data from L2.

This means that any data found in L1 is guaranteed to also exist in L2, L3, and RAM. The L1 cache contains the "VIP" data—the most strictly selected, highly active information.

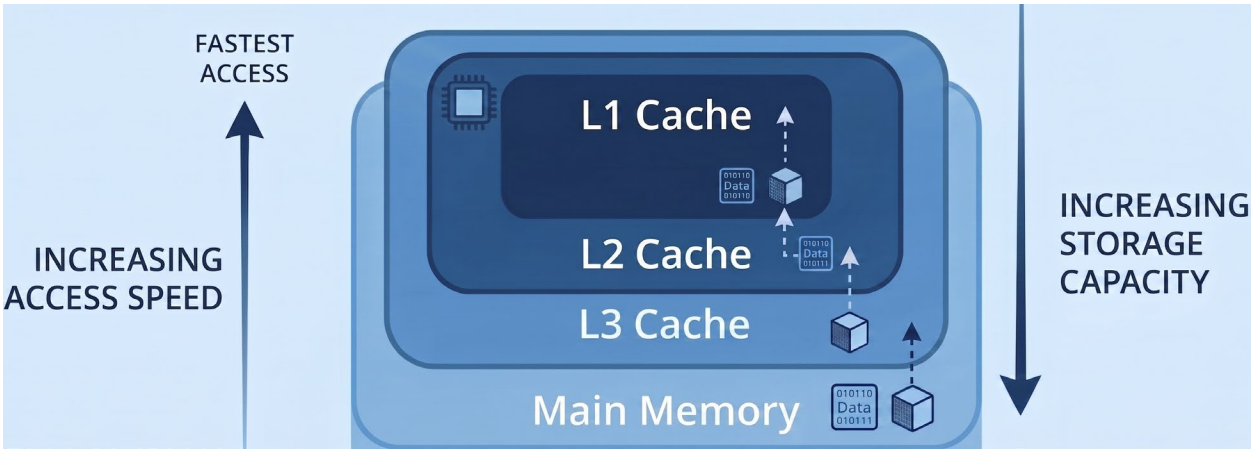


Fig 16.2: The Content of Multi-Level Caches

16.3.3 Data Movement Rules

The CPU cannot skip levels. It cannot "reach over" L1 to talk directly to L2. If L1 misses (the requested data is absent in L1), it requests from L2. If L2 misses, it requests from L3. When data returns from RAM, it is copied into L3, then L2, and finally L1.

16.4 Average Memory Access Time

We measure the efficiency of a cache system using Average Memory Access Time. This metric accounts for the high probability of finding data in fast cache versus the low probability of a slow RAM penalty.

16.4.1 Calculation Example

Scenario:

- Access Times: T_L1=1ns, T_L2=5ns, T_L3=20ns, T_Mem=60ns
- Cumulative Hit Rates: L1 (90%), L2 (95%), L3 (99%)

The Four Cases of Access:

1. Case 1 (L1 Hit): 90% of requests find data in L1.
2. Case 2 (L1 Miss, L2 Hit): 5% of requests (95% - 90%) miss L1 but hit L2.
3. Case 3 (L1/L2 Miss, L3 Hit): 4% of requests (99% - 95%) must go to L3.
4. Case 4 (All Miss): 1% of requests (100% - 99%) must go all the way to RAM.

Formula: T_avg = T_L1 + (1 - Hit_L1)*T_L2 + (1 - Hit_L2)*T_L3 + (1 - Hit_L3)*T_Mem

$$T_{average} = 1 \times T_{L1} + (1 - hit_{L1}) \times T_{L2} + (1 - hit_{L2}) \times T_{L3} + (1 - hit_{L3}) \times T_{Mem}$$

Calculation: T_avg = 1 + (0.10 * 5) + (0.05 * 20) + (0.01 * 60) = 3.1 ns

By effectively satisfying 90% of requests in 1 ns, the system brings the average time down to 3.1 ns, drastically faster than the 60 ns RAM speed.

16.5 Memory Virtualization

While Cache solves the speed problem, Physical Memory (RAM) has its own set of structural limitations regarding Security, Fragmentation, and Capacity.

16.5.1 The Problems with Physical RAM

- **Security (Isolation):** If all programs shared the same physical address space, a malicious program could theoretically calculate an address offset to read memory allocated to other programs which may contain sensitive information (e.g., passwords).

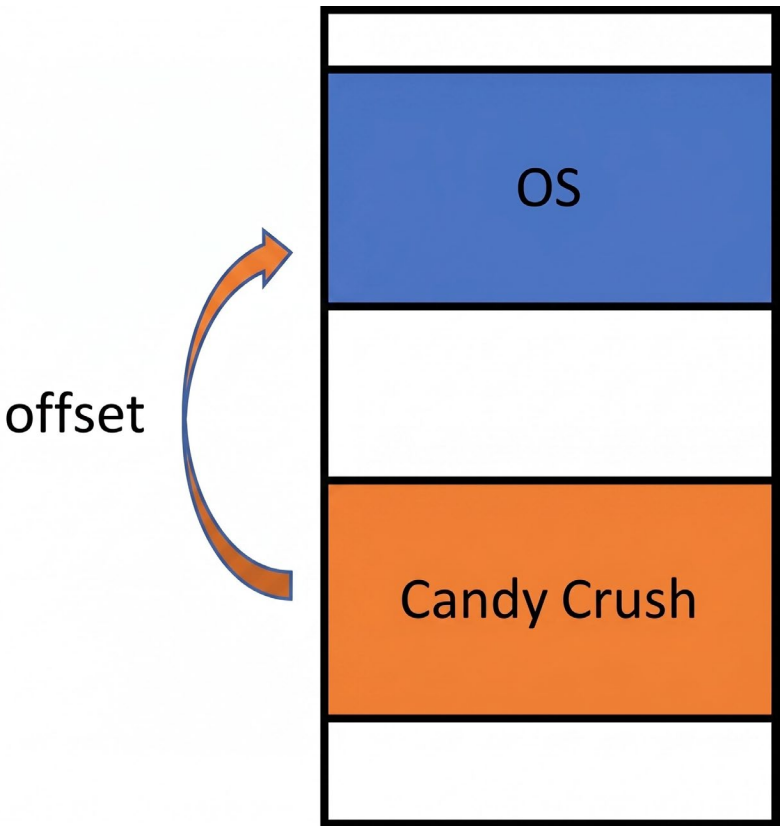


Fig 16.3: Cross-program memory access

- **Fragmentation:** Programs require consecutive blocks of memory. Imagine you have 1 GB of free space at the start of RAM and 200 MB at the end. If you try to run an application requiring 1.1 GB, it will fail to launch because the free space is not continuous.

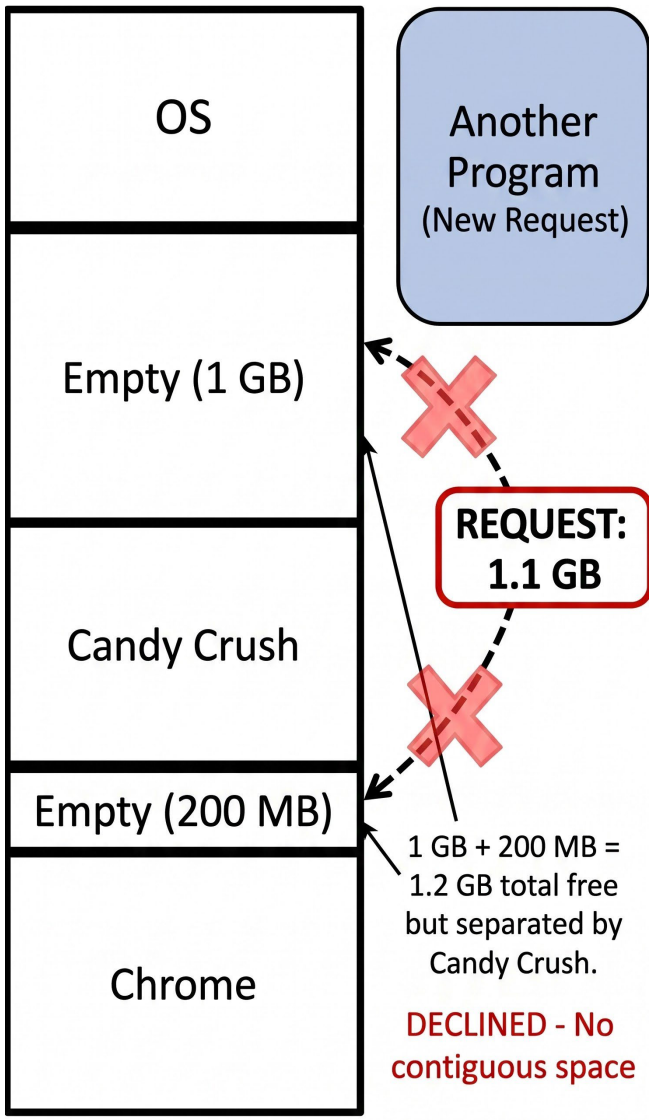


Fig 16.4: Fragmentation issue. A request for 1.1 GB from another program will be declined because there is no consecutive 1.1 GB in memory.

- **Finite Capacity:** You simply cannot run a 10 GB application on a laptop with 4 GB of RAM using only physical addressing.

16.5.2 The Solution: Virtual Memory

Memory Virtualization acts as a layer of abstraction. It assigns a "fake" (Virtual) address to the program, which the hardware then translates into a "real" (Physical) address via a Mapping Table. The OS divides memory into fixed-size chunks called Pages. Each page in a physical storage (DRAM, disk, cloud) is mapped to a virtual page in virtual memory. The

mapping table tracks the corresponding relation between physical memory page and virtual memory page.

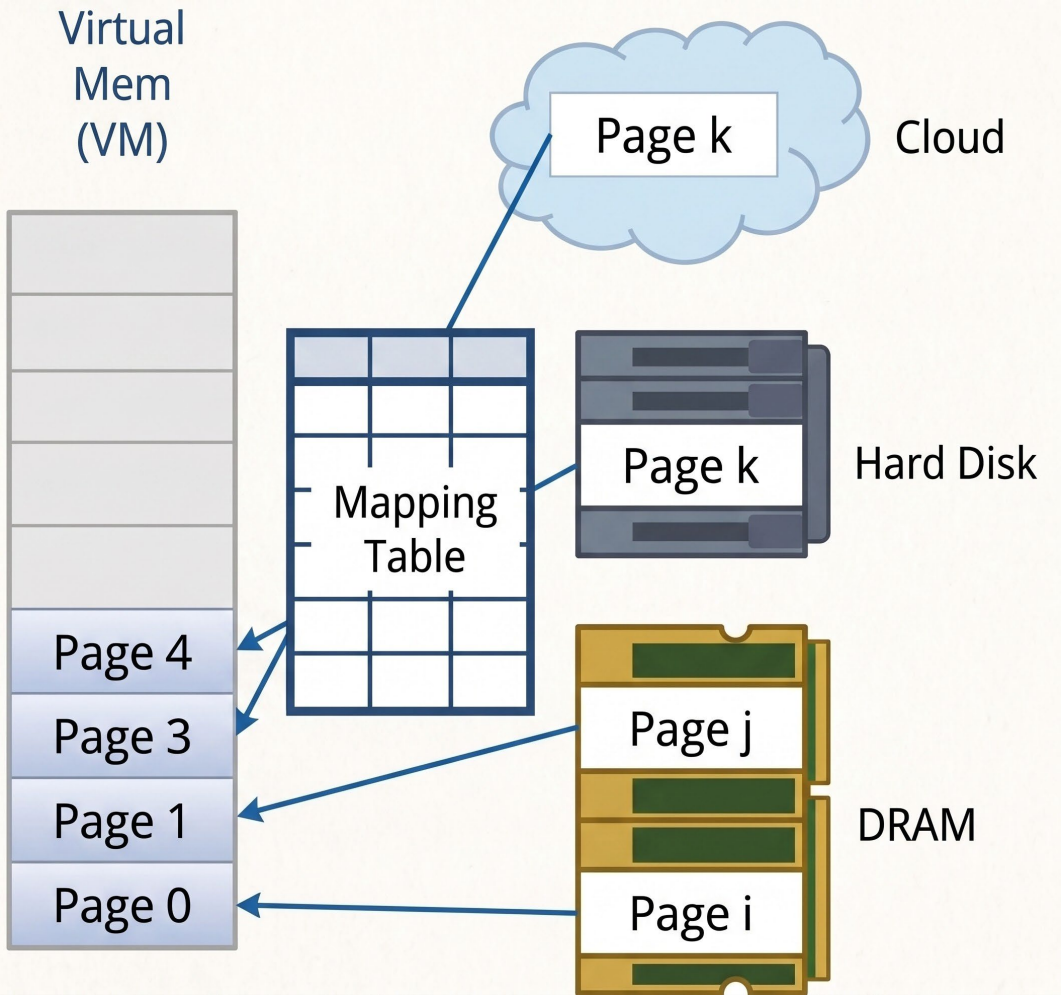


Fig 16.5: Memory Virtualization

16.5.3 Benefits of Virtualization

- **Isolation:** Virtualization gives each application its own private Mapping Table. This is like a hotel key card: even though you have a key, it is programmed to open only your room. You cannot access Room 100 if your key is mapped to Room 200.
- **Solves Fragmentation:** Virtual pages that appear consecutive to the program (Page 0, Page 1) can be mapped to physical pages scattered anywhere in RAM/Disk/Cloud (Page J, Page K).

- **Unlimited Capacity (Swapping):** If RAM is full, the OS can move inactive pages to the Hard Disk (Swap). A program can utilize 10 GB of address space by seamlessly swapping data between fast DRAM and slow Disk storage.

16.5.4 Limitation of Virtualization

- Latency: It takes a longer time to extract data from virtual memory. Mapping table queries consume time. Additionally, data reading from hard disk and cloud data server consumes more time. This is particularly challenging for applications where low latency is required.

Image Credits

This work contains copyrighted material the use of which has not always been specifically authorized by the copyright owners. The creators of this OER believe in good faith that reuse of these materials constitutes a “fair use” as described in Section 107 of the Copyright Act . Downstream users who wish to use copyrighted material from this resource for purposes beyond those authorized by the fair use doctrine should obtain permission from the copyright owner(s) of the original content.

Unless noted below, all figures used in the text are the intellectual property of the content creators.

AI Disclosure

MAVS OPEN PRESS AI POLICY

Mavs Open Press embraces innovation in open publishing, including the ethical and transparent use of generative AI. The Mavs Open Press AI Policy permits the use of AI assistance throughout the creation process and the inclusion of AI-generated images, data visualizations, graphics, and supplemental materials. Authors must disclose any instances of AI-generated content on this page and must ensure it meets all applicable copyright and licensing requirements.

AI-ASSISTED CONTENT

Mavs Open Press defines work as AI-assisted when author-created content is enhanced, organized, or edited using a generative AI tool.

This OER was created using AI assistance.

AI-GENERATED CONTENT

Mavs Open Press defines work as AI-generated when it is produced by a generative AI tool in response to a prompt from an author. AI-generated content may be in any format, including text, data, images, videos, or audio files. The content may or may not be edited or modified by the author after it is generated.

This OER contains AI-generated content. AI-generated content appears in the following chapters:

Chapter 1-Figure 1, 2 and, alt texts (Tool: Gemini)

Chapter 4- Figure 1 and alt texts (Tool: Gemini)

Chapter 5-Figure 1, 2 and, alt texts (Tool: Gemini)

Chapter 7-Figure 1, 2 and, alt texts (Tool: Gemini)

Chapter 9-Figure 1, 2, 3, 4 and, alt texts (Tool: Gemini)

Chapter 11- Figure 1 and alt texts (Tool: Gemini)

Chapter 14- Figure 1 and alt texts (Tool: Gemini)

Chapter 16-Figure 1 (Tool: figurelabs.ai), Figure 2, 3, 4 and 5 and, alt texts (Tool: Gemini)

Errata And Versioning History

The peer review process for Mavs Open Press resources varies by publication and author. We strive for transparency and describe the creation process in the front matter of each text so readers may evaluate the work on its own merit. Because we offer web-based services, we can quickly and easily address any errors that arise after publication. This page provides a record of edits and changes made to the web version of this book since its initial publication.

Changes logged here are reflected in the web text only until a new version of the resource is released, at which point the file downloads are also updated. Version updates are noted in the Status field below.

If you have a correction to suggest, submit it to digitalpublishing@uta.libanswers.com. We will contact the author then make and log necessary changes here. Below is a list of Correction Types we currently update:

1. Typo
2. Broken link
3. Addition
4. Other factual inaccuracy in content
5. Incorrect calculation or solution
6. General/pedagogical suggestion or question

Errata and Versioning History Log

Date Submitted	Format	Correction Type	Location	Description	Status